

SYSTEM DESIGN CORE

Thiết Kế Hệ Thống

Từ con số 0 đến chuyên gia

Biên soạn lại từ systemdesigncore.site — nội dung gốc của Steve Bang.

Thuật ngữ chuyên ngành được Việt hoá, kèm thuật ngữ gốc ở lần xuất hiện đầu tiên mỗi bài.

Mục lục

Lời mở đầu	System Design: Bắt đầu đúng ngay từ ngày đầu	3
Phần 0	Chuyển Đổi Mô Hình Tư Duy	8
Phần 0 · Bài 1	Từ Ưu Tiên Viết Mã đến Ưu Tiên Vấn Đề: Bước Đầu Thay Đổi Tư Duy	13
Phần 0 · Bài 2	Từ Giải Pháp Hoàn Hảo đến Tư Duy Đánh Đổi	21
Phần 0 · Bài 3	Thành Phần và Giao Tiếp: Hiểu Bản Chất Của Một Hệ Thống	35
Phần 0 · Bài 4	Tư Duy Theo Ràng Buộc: Kỹ Thuật Là Quản Lý Giới Hạn	51
Phần 0 · Bài 5	Scale Intuition: Xây Dựng Cảm Giác Về Numbers	67
Phần 0 · Bài 6	System Thinking Framework: Nhìn Hệ Thống Như Một Tổng Thể	88
Phần 0 · Bài 7	System Question Framework – Checklist Chuẩn Khi Design Hệ Thống	106
Phần 0 · Bài 8	Mental Model Recap – củng cố Tư Duy System Design	116
Phụ lục A	System Design Interview Preparation	125
Phụ lục B	Bảng thuật ngữ Anh – Việt	133

System Design: Bắt đầu đúng ngay từ ngày đầu

Nền tảng này được xây để giúp developer Việt Nam chuyển từ **code-first** sang **system thinking**.

Bạn không chỉ học cách vẽ sơ đồ hệ thống. Bạn học cách tư duy như một kỹ sư thiết kế sản phẩm thật: hiểu bài toán, nhìn trade-off, và ra quyết định kiến trúc phù hợp với bối cảnh.

Mission: Giúp bạn đi từ người "làm feature" thành người có thể thiết kế, giải thích và bảo vệ kiến trúc hệ thống trong môi trường production.

1. Bạn sẽ học được gì

- **System thinking:** nhìn hệ thống như một tổng thể thay vì từng service rời rạc.
- **Architecture mindset:** biết cách chia thành phần, ranh giới trách nhiệm, và luồng dữ liệu.
- **Trade-off reasoning:** hiểu vì sao không có "best solution", chỉ có giải pháp phù hợp.
- **Real-world system design:** học theo các tình huống gần với production: scale, reliability, observability, cost.
- **Product-aware engineering:** kết nối quyết định kỹ thuật với nhu cầu người dùng và mục tiêu sản phẩm.

2. Trang này dành cho ai

Phù hợp nhất với

- Backend engineer muốn nâng cấp tư duy thiết kế hệ thống.
- Fullstack/frontend engineer muốn hiểu kiến trúc end-to-end.

- Developer đang chuẩn bị system design interview.
- Dev từ junior đến senior muốn phát triển career dài hạn theo hướng architecture.

Có thể chưa phù hợp nếu

- Bạn chỉ muốn “template trả lời phỏng vấn” mà không muốn hiểu bản chất.
- Bạn không sẵn sàng rèn tư duy đặt câu hỏi và phân tích trade-off.

3. Triết lý học tập

Bắt đầu đơn giản, tiến hóa dần

Thiết kế hệ thống tốt không bắt đầu từ độ phức tạp. Nó bắt đầu từ phiên bản đơn giản nhất có thể chạy được, rồi mở rộng theo nhu cầu thật.

Mental model trước, công nghệ sau

Framework và tool thay đổi liên tục. Mental model về latency, consistency, throughput, fault tolerance mới là thứ giữ giá trị lâu dài.

Problem-first, không phải tool-first

Đừng hỏi "dùng Redis hay Kafka trước". Hãy hỏi: "Bài toán là gì, constraint là gì, và chi phí trade-off là gì?"

Tư duy production

Mọi quyết định đều cần nghĩ tới vận hành thật: monitor thế nào, fail thì sao, rollback ra sao, scale bao nhiêu là đủ.

4. Learning Journey: Zero → Hero

Phase	Trọng tâm	Thời lượng gợi ý
Phase 0: Mental Model Shift	Chuyển tư duy từ code-first sang problem-first, trade-off thinking	2 tuần

Phase	Trọng tâm	Thời lượng gợi ý
Phase 1: Foundation — Thinking in Systems	Components, communication patterns, data flow, bottlenecks, CAP/availability	3-4 tuần
Phase 2: Core Building Blocks	Load balancing, caching, database scaling, queue, integration trade-offs	4-5 tuần
Phase 3: Distributed Systems Fundamentals	Failure model, consistency model, consensus, distributed transaction, ID	5-6 tuần
Phase 4: Scalability & Performance	Performance, caching strategy, CDN/edge, rate limit, async, observability	4-5 tuần
Phase 5: Real-World Architecture Patterns	Evolution mindset, read-heavy system, social/feed architecture	4-5 tuần
Phase 6: System Design Mastery	Framework, SNAKE methodology, decision-making, interview strategy, architect mindset	Ongoing

5. Cách học hiệu quả

Nhịp học khuyến nghị

- 4-5 buổi/tuần
- 60-90 phút/buổi
- Mỗi tuần dành 1 buổi để review và viết lại reasoning của bản thân

Quy trình học cho mỗi lesson

1. Đọc mục tiêu và context của lesson.
2. Tự tóm tắt bài toán bằng 3-5 dòng.
3. Liệt kê 2-3 phương án khả thi.
4. Chọn 1 phương án và ghi rõ trade-off.
5. So sánh với nội dung bài học và update mental model.

Nguyên tắc quan trọng

- Học để **ra quyết định**, không phải học để nhớ keyword.

- Nếu gặp khái niệm khó, quay về nguyên lý gốc (data flow, bottleneck, constraint).
 - Viết lại bằng ngôn ngữ của bạn sau mỗi lesson để biến kiến thức thành kỹ năng.
-

6. Prerequisites

Bạn không cần biết hết distributed systems từ đầu, nhưng nên có:

- Kiến thức lập trình cơ bản (backend hoặc fullstack).
 - Hiểu HTTP/API ở mức làm sản phẩm thực tế.
 - Nền tảng database cơ bản (SQL/NoSQL ở mức sử dụng).
 - Tư duy chủ động đặt câu hỏi "vì sao".
-

7. Kết quả kỳ vọng sau lộ trình

Sau khi hoàn thành, bạn có thể:

- Phân tích đề bài system design có cấu trúc.
 - Thiết kế kiến trúc từ simple baseline đến scalable architecture.
 - Trình bày trade-off rõ ràng với team/product/stakeholder.
 - Tự tin hơn khi tham gia system design interview.
 - Ra quyết định kỹ thuật gắn với giá trị sản phẩm và mục tiêu dài hạn.
-

8. Cam kết thời gian thực tế

- Tổng thời gian để hoàn thành từ Phase 0 đến Phase 5: khoảng **5-7 tháng** nếu học đều.
 - Phase 6 là quá trình liên tục để nâng chuẩn tư duy architect-level.
 - Bạn không cần học nhanh. Bạn cần học **đúng cách và bền vững**.
-

9. Bắt đầu ngay

Nếu bạn mới vào nền tảng, hãy bắt đầu từ **Phase 0 — Mental Model Shift**.

Đây là phần quan trọng nhất để reset tư duy trước khi đi sâu vào software architecture và distributed systems.

PHẦN 0

Chuyển Đổi Mô Hình Tư Duy

Giai đoạn khởi đầu giúp bạn chuyển từ tư duy lấy mã nguồn làm trung tâm (*code-first*) sang tư duy hệ thống (*system thinking*) trước khi học sâu về thiết kế hệ thống.

1) Giới thiệu Phase

Phase 0 là cổng vào của toàn bộ hành trình System Design.

Giai đoạn này không tập trung vào tool, framework hay implementation. Mục tiêu chính là thay đổi cách bạn nhìn nhận và giải quyết vấn đề — từ **code-centric thinking** sang **system-level thinking**.

Phần lớn developer gặp khó khi học system design không phải vì thiếu kỹ thuật, mà vì chưa có mental model đúng về hệ thống. Phase này giúp xây dựng nền tảng tư duy đó trước khi đi sâu vào kiến trúc và distributed systems.

Nếu không có bước chuyển này, bạn dễ:

- Chọn công nghệ trước khi hiểu vấn đề
- Tối ưu local solution thay vì toàn hệ thống
- Bỏ qua constraint thực tế của production
- Không thấy trade-off trong quyết định kiến trúc

Phase 0 giúp bạn tránh những sai lầm này.

2) Vì sao phase này tồn tại

Coding và System Design là hai lớp tư duy khác nhau

- **Coding:** tập trung vào implementation và logic chức năng.
- **System design:** tập trung vào scale, reliability, constraint và trade-off.

Một system tốt không chỉ chạy đúng — mà phải chạy ổn định, mở rộng được và phù hợp với bối cảnh thực tế.

Engineer mindset và Architect mindset khác nhau

- **Engineer mindset:** giải quyết task trước mắt.
- **Architect mindset:** nhìn toàn hệ thống, dự đoán bottleneck và tối ưu dài hạn.

Phase này giúp bạn chuyển sang góc nhìn architect.

Trade-off là trung tâm của mọi quyết định kiến trúc

Không có giải pháp hoàn hảo.

Chỉ có giải pháp phù hợp với:

- scale
- constraint
- cost
- latency
- reliability
- business context

3) Learning Objectives

Sau phase này bạn sẽ:

- Chuyển từ code-first sang problem-first reasoning.
- Hiểu system design là quản lý constraint và trade-off.
- Hình thành trực giác về scale và bottleneck.
- Nhìn hệ thống như một tổng thể thay vì từng component riêng lẻ.
- Hiểu high-level architecture của distributed systems.
- Có framework tư duy trước khi đi vào kỹ thuật chuyên sâu.

4) Bạn sẽ học gì trong phase này

Phase này xây dựng nền tảng tư duy cốt lõi của system architect:

- **Problem-first thinking** — bắt đầu từ bài toán, không bắt đầu từ tool.
- **Trade-off mindset** — hiểu mất được và mất mát trong mỗi quyết định.
- **System components & communication** — các thành phần chính của hệ thống hiện đại.
- **Constraint thinking** — design dựa trên giới hạn thực tế.
- **Scale intuition** — hiểu hệ thống hoạt động ở quy mô lớn.
- **System thinking framework** — nhìn hệ thống như một flow hoàn chỉnh.

5) Lessons Overview

Lesson 1 – Từ Code-First đến Problem-First – Thay Đổi Tư Duy Architect

Xây nền tư duy bắt đầu từ bài toán, requirement và constraint trước khi chọn giải pháp kỹ thuật.

Lesson 2 – Từ Perfect Solution đến Trade-off Thinking – Nghệ Thuật Của Quyết Định Kiến Trúc

Hiểu rằng mọi lựa chọn kiến trúc đều có chi phí và giới hạn.
Học cách ra quyết định trong môi trường production thực tế.

Lesson 3 – Thinking in Constraints – Engineering Là Quản Lý Giới Hạn

Hiểu rằng engineering thực chất là quản lý resource và constraint: compute, memory, network và data.

Lesson 4 – Scale Intuition – Xây Dựng Cảm Giác Về Numbers

Xây dựng trực giác về traffic, data growth, throughput và bottleneck khi hệ thống mở rộng.

Lesson 5 – System Thinking Framework – Nhìn Hệ Thống Như Một Tổng Thể

Học cách nhìn hệ thống theo flow: input → processing → storage → output, thay vì từng module rời rạc.

Lesson 6 – System Question Framework - Checklist Chuẩn Khi Design Hệ Thống

Học cách đặt câu hỏi đúng khi thiết kế hệ thống.

Lesson 7 – Mental Model Recap - Củng Cố Tư Duy System Design

Tổng kết Phase 0 với 4 mindset shift quan trọng nhất: không có best solution, luôn hỏi why, trade-off quan trọng hơn technology, và system thinking trên code. Nền tảng tư duy cho system architect.

6) Kết quả sau khi hoàn thành Phase 0

Bạn có thể:

- Nhìn hệ thống dưới góc độ bài toán và constraint.
- Giải thích quyết định kiến trúc bằng trade-off thay vì cảm tính.
- Hiểu high-level architecture của một system.
- Dự đoán bottleneck và rủi ro cơ bản.
- Thảo luận system design với tư duy có cấu trúc.

Bạn sẽ không còn chỉ "viết code", mà bắt đầu **thiết kế hệ thống**.

7) Chuẩn bị cho Phase 1

Phase 0 giúp bạn có **cách nghĩ đúng**.

Phase 1 — **Foundation: Thinking in Systems** sẽ mở rộng từ mindset sang nền tảng kỹ thuật:

- system components
- communication patterns
- data flow
- latency vs throughput
- bottlenecks
- availability
- consistency
- CAP intuition

Bạn sẽ chuyển từ tư duy → kiến thức hệ thống cốt lõi.

Từ Ưu Tiên Viết Mã đến Ưu Tiên Vấn Đề: Bước Đầu Thay Đổi Tư Duy

Tôi còn nhớ rõ lần đầu tiên bị một kiến trúc sư lâu năm (*senior architect*) hỏi: "Tại sao em chọn Redis?"

Câu trả lời của tôi lúc đó? "Vì Redis nhanh ạ."

Anh ấy mỉm cười: "Nhanh hơn cái gì? Giải quyết vấn đề gì? Chi phí (*cost*) của nó là gì?"

Tôi câm nín. Đó là lúc tôi nhận ra: mình đang nghĩ như một lập trình viên, không phải một kiến trúc sư (*architect*).

Tại Sao Phải Thay Đổi Tư Duy?

Khi bạn giỏi viết mã, bạn có xu hướng nhảy thẳng vào giải pháp. Thấy bộ nhớ đệm (*cache*)? Nghĩ ngay đến Redis. Thấy hàng đợi (*queue*)? RabbitMQ. Vi dịch vụ (*Microservices*)? Kubernetes.

Nhưng đó là **tư duy ưu tiên giải pháp** (*solution-first thinking*) - tư duy nguy hiểm nhất trong thiết kế hệ thống (*system design*).

Tại sao? Vì bạn đang chọn công cụ trước khi hiểu vấn đề. Giống như mua thuốc trước khi biết mình bệnh gì.

Một kiến trúc sư lâu năm không bắt đầu bằng "Tôi sẽ dùng X". Họ bắt đầu bằng "Vấn đề tôi cần giải quyết là gì?"

Tư Duy Ưu Tiên Viết Mã (*Code-First Thinking*): Tư Duy Lập Trình Viên

Đây là cách một lập trình viên (*developer*) thường nghĩ:

Kịch bản: Trang web bị chậm

Tư duy lập trình viên:

"Trang web chậm → Cần bộ nhớ đệm → Dùng Redis → Xong!"

Nghe có vẻ hợp lý, nhưng thiếu rất nhiều thứ:

- Chậm ở đâu? Truy vấn cơ sở dữ liệu (*database queries*)? Mạng (*network*)? Lời gọi API (*API calls*)?
- Chậm khi nào? Giờ cao điểm (*peak hours*)? Mọi lúc?
- Chậm bao nhiêu? 100ms hay 5s?
- Bộ nhớ đệm thực sự giải quyết được không?
- Chi phí của bộ nhớ đệm là gì? Bộ nhớ (*memory*)? Độ phức tạp (*complexity*)? Dữ liệu cũ (*stale data*)?

Đó là 5 câu hỏi mà tư duy ưu tiên viết mã bỏ qua. Và đó chính là lý do nhiều giải pháp kỹ thuật thất bại.

Ví Dụ Thực Tế: Thảm Họa Vi Dịch Vụ

Tôi từng chứng kiến một công ty khởi nghiệp (*startup*) với 0 người dùng quyết định xây dựng hệ thống bằng vi dịch vụ từ ngày đầu.

Lý do? "Netflix dùng vi dịch vụ mà."

Kết quả sau 6 tháng:

- 15 dịch vụ (*services*) cho một tính năng đơn giản
- Triển khai (*deploy*) mất 2 giờ
- Gỡ lỗi (*debug*) là một cơn ác mộng (lỗi nằm ở dịch vụ nào?)
- Cả nhóm kiệt sức vì độ phức tạp

Họ đã giải quyết được vấn đề gì? Không có. Vì họ chưa hề có vấn đề về quy mô (*scale*). Họ cần một hệ thống nguyên khối (*monolith*) đơn giản, ra mắt nhanh, cải tiến nhanh.

Bài học: Netflix dùng vi dịch vụ để giải quyết vấn đề của Netflix (hàng triệu người dùng, hàng trăm nhóm phát triển). Công ty khởi nghiệp kia không có vấn đề đó.

Tư Duy Ưu Tiên Vấn Đề (*Problem-First Thinking*): Tư Duy Kiến Trúc Sư

Đây là cách một kiến trúc sư tiếp cận:

Kịch bản: Trang web bị chậm

Tư duy kiến trúc sư:

1. Xác định vấn đề:

- Chậm ở đâu? → Đo đạc: 80% thời gian nằm ở truy vấn cơ sở dữ liệu
- Truy vấn nào? → 3 truy vấn chậm nhất chiếm 90% tải

2. Hiểu nguyên nhân:

- Tại sao chậm? → Thiếu chỉ mục (index)
- Tại sao lại thiếu? → Thiết kế lược đồ (schema) không tối ưu cho cách truy vấn

3. Đánh giá các phương án:

- Phương án A: Thêm chỉ mục → Đơn giản, hiệu quả
- Phương án B: Lưu đệm kết quả truy vấn → Phức tạp, có thể không cần
- Phương án C: Viết lại truy vấn → Tốn thời gian

4. Chọn giải pháp:

- Bắt đầu với chỉ mục (đơn giản, tác động lớn)
- Giám sát
- Thêm bộ nhớ đệm nếu vẫn chưa đủ

Thấy sự khác biệt chưa?

Kiến trúc sư không nhảy thẳng vào Redis. Họ **đo đạc, phân tích, đánh giá, rồi mới quyết định**.

Khung tư duy (*Framework*): 5 Câu Hỏi Vàng

Trước khi chọn bất kỳ giải pháp kỹ thuật nào, hãy tự hỏi:

1. Vấn đề thực sự là gì?

Không phải "trang web chậm". Mà là "thời gian tải trang chủ là 3 giây, người dùng kỳ vọng dưới 1 giây".

Cụ thể, đo đạc được, có mục tiêu (*target*) rõ ràng.

2. Tại sao vấn đề này xảy ra?

Đào sâu. Hỏi "Tại sao?" ít nhất 3 lần.

Trang web chậm

- Tại sao? Truy vấn cơ sở dữ liệu chậm
- Tại sao? Phải quét toàn bộ bảng (full table scan)
- Tại sao? Không có chỉ mục trên cột email
- Nguyên nhân gốc: Thiếu chỉ mục

3. Giải pháp này giải quyết vấn đề gì?

"Redis làm trang web nhanh" là câu trả lời tồi.

"Redis lưu đệm kết quả truy vấn để giảm tải (*load*) cơ sở dữ liệu từ 1000 req/s xuống 100 req/s" là câu trả lời tốt.

4. Chi phí của giải pháp là gì?

Mọi giải pháp đều có chi phí:

- Redis: Thêm hạ tầng (*infrastructure*), độ phức tạp của việc vô hiệu hoá bộ nhớ đệm (*cache invalidation*), chi phí bộ nhớ
- Ví dụ: Độ trễ (*latency*) mạng, độ phức tạp của hệ thống phân tán (*distributed system*), gỡ lỗi khó hơn
- Phân mảnh dữ liệu (*Sharding*): Truy vấn xuyên nhiều mảnh rất khó, phân mảnh lại (*resharding*) là cơn ác mộng

Không có bữa trưa nào miễn phí. Sự đánh đổi (*Trade-off*) luôn luôn tồn tại.

5. Có cách đơn giản hơn không?

Đây là câu hỏi quan trọng nhất.

Trước khi thêm Redis, bạn đã thử thêm chỉ mục chưa? Trước khi chuyển sang vi dịch vụ, bạn đã thử mở rộng hệ thống nguyên khối chưa?

"Giải pháp đơn giản trước. Giải pháp phức tạp khi thật sự cần."

Ví Dụ So Sánh: Thiết Kế Tính Năng "Đếm Lượt Xem"

Hãy xem hai cách tiếp cận (*approach*) khác nhau cho cùng một tính năng.

Cách tiếp cận 1: Ưu Tiên Viết Mã (Tệ)

Tính năng: Đếm số lượt xem video

Tư duy lập trình viên:

"Cần đếm lượt xem → Dùng bộ đếm Redis → Tăng mỗi lượt xem → Xong"

Cài đặt:

- Mỗi lượt xem → `redis.incr('video:123:views')`
- Hiển thị: `redis.get('video:123:views')`

Trông ổn đấy. Nhưng sau 1 tháng chạy trên môi trường thực tế (*production*):

Vấn đề phát sinh:

- Máy chủ (*server*) Redis khởi động lại → Mất hết dữ liệu (lượt xem về 0)
- Không có dữ liệu lịch sử (*historical data*) (lượt xem theo ngày/tháng)
- Không theo dõi được xu hướng (*trending*) (video có lượt xem tăng nhanh)
- Lưu lượng (*traffic*) từ bot spam tăng bộ đếm

Lập trình viên bây giờ phải tái cấu trúc (*refactor*) toàn bộ, di chuyển dữ liệu (*migrate data*), xử lý thời gian ngừng hoạt động (*downtime*).

Cách tiếp cận 2: Ưu Tiên Vấn Đề (Tốt)

Tính năng: Đếm số lượt xem video

Tư duy kiến trúc sư:

1. Yêu cầu:

- Độ chính xác: Có cần chính xác 100%? (Có lẽ là không)
- Quy mô: Bao nhiêu video? Bao nhiêu lượt xem/giây?
- Phân tích: Có cần theo dõi theo thời gian không?
- Gian lận: Có cần chặn bot spam không?

2. Thiết kế:

- Ghi vào cơ sở dữ liệu (lưu bền vững, là nguồn dữ liệu gốc)
- Ghi theo lô bất đồng bộ (gom 10 giây) để giảm tải CSDL
- Lưu đệm trong Redis (đọc nhanh, có thể sai lệch 1-2%)
- Tác vụ nền đồng bộ Redis ↔ CSDL mỗi phút

3. Các đánh đổi chấp nhận:

- Số lượt xem có thể trễ vài giây (chấp nhận được)
- Có thể sai lệch nhỏ (chấp nhận được với chỉ số mạng xã hội)
- Phức tạp hơn giải pháp 1 (cần thiết để đảm bảo độ tin cậy)

Cách tiếp cận này xử lý được:

- Redis gặp sự cố → Chuyển sang đọc từ cơ sở dữ liệu
- Dữ liệu lịch sử → Có sẵn trong cơ sở dữ liệu
- Xu hướng → Truy vấn cơ sở dữ liệu theo khoảng thời gian
- Bot spam → Giới hạn tần suất (*rate limit*) + lớp kiểm tra hợp lệ

Bài Tập Thực Hành: Thay Đổi Tư Duy (*Mindset*)

Bài tập 1: Phân Tích Quyết Định Của Bạn

Nhớ lại một quyết định kỹ thuật gần đây bạn đã làm. Trả lời:

1. **Bạn chọn giải pháp gì?** (VD: Dùng MongoDB thay vì PostgreSQL)
2. **Lý do bạn chọn?** Viết ra lý do thật sự, không phải lý do "đẹp".
3. **Bạn đã đo đạc vấn đề chưa?** Có số liệu đo (*metrics*) cụ thể không?
4. **Bạn có xem xét các phương án thay thế** (*alternatives*) **không?** Có so sánh với PostgreSQL không?
5. **Chi phí và đánh đổi là gì?** Bạn đã nghĩ đến chưa?

Nếu câu trả lời cho câu 3, 4, 5 là "không" → Đó là tư duy ưu tiên viết mã.

Bài tập 2: Phân Tích Ngược (*Reverse Engineering*)

Chọn một app bạn hay dùng (Facebook, Instagram, Shopee...).

Nhiệm vụ: Dùng đoán họ dùng công nghệ gì. Thay vào đó:

1. **Xác định các vấn đề họ phải giải quyết:**
 - Quy mô: Bao nhiêu người dùng?
 - Tính nhất quán (*Consistency*): Số lượt thích có cần chính xác tuyệt đối không?
 - Độ trễ: Bảng tin (*feed*) cần tải nhanh đến mức nào?
2. **Lập luận:**
 - Với quy mô này, họ không thể chỉ dùng 1 cơ sở dữ liệu
 - Với yêu cầu về độ trễ, họ phải có lớp bộ nhớ đệm (*cache layer*)
 - Với yêu cầu về tính nhất quán, họ có thể chấp nhận nhất quán cuối cùng (*eventual consistency*)

3. Dự đoán giải pháp:

- Có lẽ dùng CDN cho tài nguyên tĩnh (*static assets*)
- Có lẽ dùng bộ nhớ đệm cho bảng tin
- Có lẽ dùng nhất quán cuối cùng cho số lượt thích

Thấy sự khác biệt chưa? Bạn đang lập luận từ vấn đề → giải pháp, không phải ngược lại.

Danh Sách Tự Kiểm Tra: Bạn Đã Thay Đổi Tư Duy Chưa?

Đánh dấu những câu bạn đồng ý:

- ☐ Tôi hỏi "Vấn đề là gì?" trước khi nghĩ giải pháp
- ☐ Tôi đo đạc vấn đề trước khi tối ưu (*optimize*)
- ☐ Tôi có thể giải thích "Tại sao" cho mọi quyết định kỹ thuật
- ☐ Tôi xem xét ít nhất 2-3 phương án thay thế trước khi chọn
- ☐ Tôi hiểu các đánh đổi của giải pháp tôi chọn
- ☐ Tôi bắt đầu đơn giản, chỉ thêm độ phức tạp khi cần thiết
- ☐ Tôi có thể bảo vệ quyết định của mình bằng dữ liệu và lập luận

Nếu bạn đánh dấu được 5/7 → Bạn đang đi đúng hướng.

Những Điểm Cốt Lõi

Tư duy ưu tiên vấn đề không phải là biết thật nhiều công nghệ.

Nó về việc:

- Hiểu vấn đề sâu sắc
- Đo đạc và phân tích
- Cân nhắc các đánh đổi
- Chọn giải pháp phù hợp (không phải "tốt nhất", mà là "vừa vặn")

Tư duy ưu tiên viết mã là phản xạ: Thấy vấn đề → Nghĩ ngay đến giải pháp quen thuộc.

Tư duy ưu tiên vấn đề là có chủ đích: Hiểu vấn đề → Phân tích → Đánh giá các phương án → Quyết định.

Sự khác biệt này quyết định bạn là một lập trình viên giỏi hay một kiến trúc sư giỏi.

Câu hỏi quan trọng nhất mà một architect luôn tự hỏi:

"Tôi đang giải quyết vấn đề gì? Và đây có thực sự là cách tốt nhất không?"

Bắt đầu hỏi câu này từ hôm nay. Mọi thứ sẽ thay đổi.

Từ Giải Pháp Hoàn Hảo đến Tư Duy Đánh Đổi

Có một câu hỏi mà mọi kỹ sư mới vào nghề (*junior engineer*) đều từng hỏi tôi: "Cách tốt nhất để thiết kế hệ thống này là gì?"

Câu trả lời của tôi luôn làm họ thất vọng: "Không có 'cách tốt nhất'. Chỉ có cách phù hợp nhất với các ràng buộc (*constraints*) của bạn."

Đó là bài học khó nhất mà mọi kiến trúc sư (*architect*) phải học: **không có giải pháp hoàn hảo.**

Tại Sao "Giải Pháp Hoàn Hảo" Không Tồn Tại?

Khi tôi mới bắt đầu, tôi tin rằng với đủ thời gian và kiến thức, tôi có thể tìm ra "kiến trúc hoàn hảo" - một kiến trúc không có nhược điểm.

Sau 10 năm thiết kế hệ thống, tôi nhận ra một sự thật tàn khốc:

Mọi quyết định kiến trúc đều là một sự đánh đổi.

Không phải vì bạn không đủ giỏi. Mà vì trong thế giới thực, bạn luôn phải chọn giữa những thứ mâu thuẫn nhau:

- **Hiệu năng** (*Performance*) **hay Tính nhất quán** (*Consistency*): Nhanh hay chính xác?
- **Khả năng mở rộng** (*Scalability*) **hay Sự đơn giản**: Dễ mở rộng hay dễ bảo trì (*maintain*)?
- **Tính linh hoạt hay Hiệu năng**: Linh hoạt hay tối ưu?
- **Chi phí** (*Cost*) **hay Độ tin cậy** (*Reliability*): Rẻ hay ổn định?

Bạn không thể có tất cả. Và đó không phải là vấn đề. Đó là thực tế.

Huyền Thoại Về "Thực Hành Tốt Nhất" (Best Practices)"

Tôi thường nghe các lập trình viên (*developers*) nói: "Vi dịch vụ (*Microservices*) là thực hành tốt nhất" hoặc "NoSQL tốt hơn SQL".

Không. Sai hoàn toàn.

Vi dịch vụ là thực hành tốt nhất cho Netflix với 200 triệu người dùng và 500 nhóm kỹ sư. Nó là **thực hành tệ nhất** cho một công ty khởi nghiệp (*startup*) 5 người với 100 người dùng.

NoSQL tốt cho Facebook với hàng tỷ bài đăng không cần quan hệ chặt chẽ. Nó là **lựa chọn tồi tệ** cho hệ thống ngân hàng cần giao dịch (*transactions*) ACID.

"Thực hành tốt nhất" không tồn tại trong chân không. Bối cảnh (*Context*) **quyết định tất cả.**

Ví Dụ Thực Tế: Hệ Thống Nguyên Khối (*Monolith*) hay Vi Dịch Vụ

Hãy phân tích sự đánh đổi (*trade-off*) của hai cách tiếp cận (*approaches*) này.

Hệ thống nguyên khối:

Ưu điểm:

- Triển khai đơn giản (1 ứng dụng)
- Gỡ lỗi dễ dàng (1 kho mã nguồn)
- Không có độ trễ mạng giữa các thành phần
- Giao dịch đơn giản (cùng một cơ sở dữ liệu)
- Phát triển nhanh ở giai đoạn đầu

Nhược điểm:

- Khó mở rộng độc lập từng phần
- Triển khai kiểu "tất cả hoặc không có gì"
- Các nhóm xung đột trên cùng kho mã nguồn
- Bị khoá chặt vào một công nghệ
- Phải khởi động lại toàn bộ ứng dụng khi cập nhật

Vi dịch vụ:

Ưu điểm:

- Mở rộng độc lập
- Triển khai độc lập
- Đa dạng công nghệ
- Các nhóm tự chủ
- Cô lập lỗi

Nhược điểm:

- Độ trễ mạng
- Giao dịch phân tán là cơn ác mộng
- Triển khai phức tạp
- Giám sát phức tạp
- Thách thức về tính nhất quán dữ liệu

Câu hỏi không phải là "cái nào tốt hơn?". Câu hỏi là:

"Với bối cảnh hiện tại của tôi, tôi cần những ưu điểm nào và có thể chấp nhận những nhược điểm nào?"

Khung tư duy (Framework): Ma Trận Đánh Đổi

Đây là khung tư duy tôi dùng để đánh giá mọi quyết định kiến trúc.

Bước 1: Xác Định Các Ràng Buộc Của Bạn

Trước khi so sánh giải pháp, hãy hiểu rõ các ràng buộc của bạn:

Ràng buộc kỹ thuật:

- Quy mô (Scale) hiện tại: 1 nghìn người dùng hay 100 triệu người dùng?
- Tốc độ tăng trưởng: gấp 2 lần/năm hay gấp 10 lần/tháng?
- Quy mô nhóm: 2 lập trình viên hay 50 lập trình viên?
- Yêu cầu về độ trễ (latency): dưới 100ms hay dưới 1 giây là ổn?

Ràng buộc kinh doanh:

- Ngân sách: Không giới hạn hay eo hẹp?
- Thời gian ra thị trường: Ra mắt trong 1 tháng hay 6 tháng?
- Quy định pháp lý: GDPR, tuân thủ PCI?

Ràng buộc về nhóm:

- Chuyên môn: Nhóm biết những gì?

- Thời gian học: Có thời gian để học không?
- Năng lực vận hành: Ai sẽ bảo trì?

Bước 2: Liệt Kê Các Đánh Đổi Một Cách Rõ Ràng

Với mỗi phương án, viết ra rõ ràng:

Phương án A: PostgreSQL

Được:

- Đảm bảo ACID
- Khả năng truy vấn phong phú (JOIN, tổng hợp)
- Hệ sinh thái trưởng thành
- Nhóm đã biết

Mất:

- Giới hạn khi mở rộng theo chiều dọc
- Thay đổi lược đồ (schema) tốn kém
- Không tối ưu cho dữ liệu phi cấu trúc

Phương án B: MongoDB

Được:

- Mở rộng theo chiều ngang dễ hơn
- Lược đồ linh hoạt
- Phù hợp để cải tiến nhanh

Mất:

- Không hỗ trợ JOIN (phải xử lý ở tầng ứng dụng)
- Độ phức tạp của nhất quán cuối cùng
- Nhóm phải học mới

Bước 3: Đối Chiếu Với Các Ưu Tiên

Xếp hạng các ưu tiên của dự án:

Ưu tiên 1: Ra mắt nhanh (MVP trong 2 tháng)

Ưu tiên 2: Năng suất của nhóm (ít phải học thêm)

Ưu tiên 3: Toàn vẹn dữ liệu (dữ liệu tài chính)

Với các ưu tiên này → PostgreSQL thắng

Tại sao? Nhóm đã biết, ACID là then chốt, có thể mở rộng sau.

Nếu các ưu tiên thay đổi:

Ưu tiên 1: Mở rộng lên 100 triệu người dùng (đã đạt PMF)
Ưu tiên 2: Xử lý dữ liệu phi cấu trúc (nội dung do người dùng tạo)
Ưu tiên 3: Hiệu năng đọc

Với các ưu tiên này → MongoDB có thể thắng
Tại sao? Khả năng mở rộng + tính linh hoạt quan trọng hơn hạn chế về JOIN.

Thấy chưa? Cùng 2 phương án, khác ưu tiên → khác quyết định.

Tình Huống Thực Tế: Có Dùng Bộ Nhớ Đệm (Cache) Hay Không?

Hãy xem một quyết định tưởng chừng đơn giản: "Có nên dùng bộ nhớ đệm không?"

Hầu hết lập trình viên nghĩ: "Bộ nhớ đệm làm nhanh hơn → Lúc nào cũng nên dùng!"

Sai rồi. Hãy phân tích các đánh đổi.

Kịch bản: Trang Sản Phẩm Thương Mại Điện Tử

Không có bộ nhớ đệm:

Yêu cầu của người dùng → Truy vấn CSDL → Trả về

Ưu điểm:

- Dữ liệu luôn mới nhất (giá, tồn kho)
- Logic đơn giản (không cần vô hiệu hoá bộ nhớ đệm)
- Không tốn thêm hạ tầng

Nhược điểm:

- Tải lên CSDL cao
- Phản hồi chậm (mỗi yêu cầu đều truy vấn CSDL)
- Khó mở rộng khả năng đọc

Có bộ nhớ đệm (Redis):

Yêu cầu của người dùng → Kiểm tra Redis → Nếu không có: Truy vấn CSDL → Trả về

Ưu điểm:

- Phản hồi nhanh ($< 10\text{ms}$)
- Giảm tải cho CSDL
- Dễ mở rộng khả năng đọc

Nhược điểm:

- Nguy cơ dữ liệu cũ (người dùng thấy sản phẩm đã hết hàng vẫn còn hàng)
- Độ phức tạp của việc vô hiệu hoá bộ nhớ đệm
- Chi phí hạ tầng (máy chủ Redis)
- Giới hạn bộ nhớ

Quyết Định Đánh Đổi

Nếu bạn là Amazon:

→ **Dùng bộ nhớ đệm** vì:

- Hàng triệu yêu cầu/giây → CSDL không chịu nổi
- Dữ liệu hơi cũ một chút vẫn chấp nhận được (số tồn kho có thể sai 1-2 sản phẩm)
- Có ngân sách cho hạ tầng
- Nhóm có chuyên môn

Nếu bạn là công ty khởi nghiệp nhỏ:

→ **Bỏ qua bộ nhớ đệm** vì:

- 100 yêu cầu/giây → CSDL xử lý được
- Dữ liệu mới nhất quan trọng hơn (tồn kho ít)
- Tiết kiệm chi phí hạ tầng
- Giữ mọi thứ đơn giản

Cùng một vấn đề, khác bối cảnh, khác quyết định.

Những Đánh Đổi Phổ Biến Mọi Kiến Trúc Sư Phải Biết

1. Tính Nhất Quán hay Tính Sẵn Sàng (*Availability*) (Định lý CAP)

Bạn không thể có cả hai trong một hệ thống phân tán (*distributed system*) khi xảy ra phân vùng mạng (*network partition*).

Chọn tính nhất quán (CP):

Ngân hàng, thanh toán, quản lý tiền kho
→ Đúng quan trọng hơn nhanh
→ Chấp nhận ngừng hoạt động nếu cần để đảm bảo tính đúng đắn

Chọn tính sẵn sàng (AP):

Bảng tin mạng xã hội, đếm lượt xem, gợi ý nội dung
→ Nhanh quan trọng hơn chính xác tuyệt đối
→ Nhất quán cuối cùng là chấp nhận được

Tư duy đánh đổi:

- Tính năng này CÓ CẦN nhất quán mạnh (*strong consistency*) không? (Thường là KHÔNG)
- Người dùng có bị ảnh hưởng nếu dữ liệu sai trong vài giây không?
- Chi phí để đảm bảo tính nhất quán là gì?

2. Chuẩn Hoá (*Normalization*) hay Phi Chuẩn Hoá (*Denormalization*)

Cơ sở dữ liệu chuẩn hoá:

```
-- 3 tables, proper 3NF
users: id, name
posts: id, user_id, content
likes: id, post_id, user_id

-- Query post with like count
SELECT p.*, COUNT(l.id) as likes
FROM posts p
LEFT JOIN likes l ON p.id = l.post_id
WHERE p.id = 123
GROUP BY p.id;
```

Toàn vẹn dữ liệu, không trùng lặp Truy vấn chậm (JOIN tốn kém)

Phi chuẩn hoá:

```
-- 1 table
posts: id, user_id, content, like_count

-- Query post with like count
SELECT * FROM posts WHERE id = 123;
```

Đọc nhanh (không cần JOIN) Nguy cơ thiếu nhất quán, tốn công khi cập nhật

Khi nào nên phi chuẩn hoá?

- Đọc >> Ghi (99% đọc, 1% ghi)
- Hiệu năng là yếu tố sống còn
- Có thể chấp nhận thiếu nhất quán đôi chút

Khi nào nên giữ chuẩn hoá?

- Ghi nhiều
- Toàn vẹn dữ liệu là yếu tố sống còn
- Quan hệ dữ liệu phức tạp

3. Xử Lý Bất Đồng Bộ (Async) hay Đồng Bộ (Sync)

Đồng bộ:

```
def upload_video(video):
    save_to_s3(video)          # 2s
    generate_thumbnail(video)  # 3s
    transcode_720p(video)     # 30s
    send_notification()        # 1s
    return "Success"          # User waits 36s
```

Đơn giản, phản hồi ngay lập tức Trải nghiệm người dùng kém, nguy cơ hết thời gian chờ (timeout)

Bất đồng bộ:

```
def upload_video(video):
    save_to_s3(video)          # 2s
    queue.add_job('process_video', video.id)
    return "Success"          # User waits 2s

# Background worker processes rest
```

Trải nghiệm người dùng tốt hơn, chịu được đột biến tải (spikes) Phức tạp, xử lý trễ, khó xử lý lỗi hơn

Quyết định đánh đổi:

- Người dùng có cần kết quả ngay không?
- Tiến trình có thể thất bại và thử lại (retry) được không?
- Nhóm có kinh nghiệm với các mẫu bất đồng bộ không?

Nghệ Thuật Của "Đủ Tốt"

Một trong những kỹ năng quan trọng nhất của kiến trúc sư: **Biết khi nào dừng tối ưu** (optimize).

Tôi từng thấy kỹ sư bỏ ra 2 tuần để tối ưu một truy vấn từ 50ms xuống 20ms.

Tác động? Người dùng không thấy khác biệt (ngưỡng cảm nhận của con người là khoảng 100ms).

Chi phí? 2 tuần làm việc = các tính năng khác bị trì hoãn.

"Hoàn hảo là kẻ thù của tốt."

Quy Tắc 80/20 Trong Kiến Trúc

20% công sức thường giải quyết 80% vấn đề.

Ví dụ: Tối Ưu Thời Gian Phản Hồi API

Hiện tại: 800ms
Mục tiêu: < 200ms

Phương án A: Thêm chỉ mục cho cơ sở dữ liệu

- Công sức: 2 giờ
- Kết quả: 800ms → 300ms (cải thiện 62%)

Phương án B: Thêm và triển khai bộ nhớ đệm Redis

- Công sức: 1 tuần
- Kết quả: 300ms → 150ms (cải thiện thêm 50%)

Phương án C: Chuyển sang Cassandra + viết lại truy vấn

- Công sức: 2 tháng
- Kết quả: 150ms → 100ms (cải thiện thêm 33%)

Quyết định khôn ngoan:

- Bắt đầu với Phương án A (hiệu quả đầu tư cao)
- Giám sát (*Monitor*)
- Phương án B nếu 300ms vẫn chưa đủ
- Có lẽ sẽ không bao giờ làm Phương án C (lợi ích giảm dần)

Bài Tập: Phân Tích Đánh Đổi

Bài tập 1: Thiết Kế Xác Thực (*Authentication*) Người Dùng

Yêu cầu: Hệ thống đăng nhập người dùng

Phương án A: JWT (**Phi trạng thái** (*Stateless*))

Liệt kê các đánh đổi:

- Được: ?
- Mất: ?
- Phù hợp nhất cho: ?

Phương án B: Phiên (*Session*) (**Có trạng thái** (*Stateful*))

Liệt kê các đánh đổi:

- Được: ?
- Mất: ?

- Phù hợp nhất cho: ?

Viết ra lập luận của bạn. Không có đúng/sai, chỉ có phù hợp/không phù hợp.

Bài tập 2: Tình Huống Thực Tế

Bối cảnh:

- Công ty khởi nghiệp, 5 kỹ sư
- Hiện tại: 10 nghìn người dùng
- Dự kiến: 100 nghìn người dùng trong 6 tháng
- Tính năng: Trò chuyện thời gian thực (*real-time*)

Các phương án:

1. Tự xây máy chủ WebSocket
2. Dùng Firebase Realtime Database
3. Dùng Twilio/Stream Chat (dịch vụ bên thứ ba)

Nhiệm vụ của bạn:

1. Liệt kê các đánh đổi của mỗi phương án
2. Xếp hạng các ưu tiên (thời gian, chi phí, khả năng mở rộng, quyền kiểm soát)
3. Đưa ra quyết định kèm lập luận

Không có "đáp án đúng". Hãy rèn luyện tư duy đánh đổi.

Các Mô Hình Tư Duy (*Mental Models*) để Đưa Ra Quyết Định

Mô hình 1: Khả Năng Đảo Ngược

Câu hỏi: Quyết định này có dễ thay đổi sau không?

Dễ đảo ngược (thay đổi dễ dàng):

- Lựa chọn bộ nhớ đệm (Redis → Memcached)
- Thư viện ORM
- Khung phát triển giao diện người dùng

→ Đừng nghĩ quá nhiều. Chọn nhanh, cải tiến sau.

Khó đảo ngược (thay đổi tốn kém):

- Lựa chọn cơ sở dữ liệu (SQL → NoSQL)
- Hệ thống nguyên khối → Vi dịch vụ
- Nhà cung cấp điện toán đám mây

→ Suy nghĩ kỹ. Cái giá của quyết định sai rất cao.

Mô hình 2: Tác Động và Công Sức

Tác động cao, Công sức thấp → LÀM NGAY
Tác động cao, Công sức cao → LẬP KẾ HOẠCH & ƯU TIÊN
Tác động thấp, Công sức thấp → LÀM NẾU CÓ THỜI GIAN
Tác động thấp, Công sức cao → KHÔNG LÀM

Ví dụ:

- Thêm chỉ mục cơ sở dữ liệu: Tác động cao, Công sức thấp → LÀM NGAY
- Viết lại thành vi dịch vụ: Tác động cao, Công sức cao → LẬP KẾ HOẠCH cẩn thận
- Tái cấu trúc (*Refactor*) tên biến: Tác động thấp, Công sức thấp → LÀM nếu có thời gian
- Tự xây hệ thống xác thực: Tác động thấp, Công sức cao → KHÔNG LÀM (dùng Auth0)

Mô hình 3: Giảm Thiểu Hối Tiếc

Câu hỏi: 6 tháng sau, quyết định nào tôi sẽ hối hận nhất?

Kịch bản: Chọn bộ công nghệ (*tech stack*) cho MVP của công ty khởi nghiệp

Quyết định A: Dùng framework mới đang thịnh hành (Deno, Bun...)

- Nguy cơ: Hệ sinh thái chưa trưởng thành, nhóm phải học, nguy cơ thất bại
- Khả năng hối tiếc: CAO (nếu thất bại, mất thời gian quý báu)

Quyết định B: Dùng bộ công nghệ "nhàm chán" đã được kiểm chứng (Node.js + Express + PostgreSQL)

- Nguy cơ: Không "ngầu", có thể bỏ lỡ tính năng mới
- Khả năng hối tiếc: THẤP (dù có chậm, nó vẫn chạy ổn định)

→ Với MVP của công ty khởi nghiệp: **Giảm thiểu hối tiếc** → **Chọn B**

Danh Sách Tự Kiểm Tra: Bạn Đã Tư Duy Theo Đánh Đổi Chưa?

Trước khi chốt bất kỳ quyết định kiến trúc nào:

- ☐ Tôi đã liệt kê ít nhất 2-3 phương án thay thế (*alternatives*)
- ☐ Tôi đã viết ra các đánh đổi (được và mất) cho mỗi phương án
- ☐ Tôi hiểu rõ bối cảnh và các ràng buộc hiện tại
- ☐ Tôi đã xếp hạng các ưu tiên rõ ràng
- ☐ Tôi có thể giải thích "tại sao phương án này PHÙ HỢP" (không phải "tốt nhất")
- ☐ Tôi đã cân nhắc khả năng đảo ngược của quyết định
- ☐ Tôi biết mình đang hy sinh cái gì để đổi lấy cái gì

Nếu đánh dấu được 6/7 → Bạn đang tư duy như một kiến trúc sư.

Những Điểm Cốt Lõi

Không có giải pháp hoàn hảo, chỉ có sự đánh đổi tối ưu.

Mọi quyết định kiến trúc là một sự đánh đổi. Công việc của kiến trúc sư không phải là tìm "thực hành tốt nhất", mà là:

1. **Hiểu rõ bối cảnh:** Quy mô, nhóm, ràng buộc
2. **Liệt kê đánh đổi một cách rõ ràng:** Được và Mất
3. **Đối chiếu với ưu tiên:** Tác động và Công sức
4. **Ra quyết định có cơ sở:** "Phù hợp" chứ không phải "hoàn hảo"
5. **Chấp nhận hệ quả:** Đón nhận cái giá của lựa chọn

Câu nói yêu thích của tôi:

"Không có giải pháp. Chỉ có sự đánh đổi." — Thomas Sowell

Một kiến trúc sư lâu năm giỏi không phải là người biết mọi mẫu thiết kế (*pattern*). Mà là người biết **mẫu nào hợp với bối cảnh nào, và tại sao.**

Đủ tốt, đã ra mắt và được cải tiến liên tục còn hơn hoàn hảo nhưng không bao giờ ra mắt.

Hãy bắt đầu nghĩ theo sự đánh đổi từ hôm nay. Mỗi khi bạn nói "giải pháp tốt nhất", hãy dừng lại và hỏi:

"Tốt nhất cho ai? Trong bối cảnh nào? Tôi đang đánh đổi cái gì?"

Thành Phần (Components) và Giao Tiếp (Communication): Hiểu Bản Chất Của Một Hệ Thống

Lần đầu tiên tôi phải gỡ lỗi (*debug*) một sự cố trên môi trường thực tế (*production*), tôi hoàn toàn choáng váng.

Phản hồi API chậm. Nhưng chậm ở đâu? Cơ sở dữ liệu (*Database*)? Ứng dụng? Mạng (*Network*)? Bộ nhớ đệm (*Cache*)?

Một kiến trúc sư lâu năm (*senior architect*) ngồi bên cạnh, vẽ một sơ đồ đơn giản:

```
Máy khách → Bộ cân bằng tải → Máy chủ ứng dụng → Bộ nhớ đệm → Cơ sở dữ liệu
```

Rồi anh hỏi: "Yêu cầu (*Request*) đi qua bao nhiêu bước? Mỗi bước mất bao lâu?"

Đó là lần đầu tiên tôi hiểu: **Hệ thống không phải là một khối mã. Nó là tập hợp các thành phần tương tác với nhau.**

Tại Sao Phải Hiểu Các Thành Phần?

Khi bạn viết mã, bạn nhìn vào hàm, lớp, mô-đun. Đó là **góc nhìn ở cấp độ mã nguồn**.

Khi bạn thiết kế hệ thống, bạn phải nhìn vào **các thành phần và cách chúng tương tác**. Đó là **góc nhìn ở cấp độ kiến trúc**.

Sự khác biệt?

Cấp độ mã nguồn:

```
def get_user(user_id):  
    return db.query("SELECT * FROM users WHERE id = ?", user_id)
```

Cấp độ kiến trúc:

Máy khách → API Gateway → Dịch vụ Người dùng → Cơ sở dữ liệu
(50ms) (10ms) (200ms)

Tổng độ trễ: 260ms

Điểm nghẽn: Cơ sở dữ liệu (200ms = 77% tổng thời gian)

Góc nhìn cấp độ kiến trúc giúp bạn:

- Nhanh chóng xác định điểm nghẽn (*bottlenecks*)
- Hiểu các điểm có thể gặp sự cố
- Thiết kế để mở rộng (*scale*)
- Trao đổi với nhóm

Bạn không thể tối ưu (*optimize*) cái bạn không hiểu.

Giải Phẫu Một Hệ Thống Web

Mọi hệ thống web, dù đơn giản hay phức tạp, đều có cùng những khối xây dựng (*building blocks*) cơ bản.

Thành phần 1: Máy Khách (*Client*) (Trình duyệt/Ứng dụng di động)

Vai trò: Giao diện người dùng, gửi yêu cầu, hiển thị phản hồi (*responses*)

Đặc điểm:

- Không đáng tin cậy (người dùng có thể sửa mã)
- Không kiểm soát được môi trường (mạng, thiết bị)
- Có trạng thái (*Stateful*) (lưu dữ liệu cục bộ)

Hàm ý cho kiến trúc sư (*architect*):

KHÔNG BAO GIỜ tin máy khách

- Chỉ kiểm tra hợp lệ ở máy khách là chưa đủ
- Không lưu dữ liệu nhạy cảm
- Không dựa vào logic phía máy khách cho các quy tắc nghiệp vụ

LUÔN kiểm tra hợp lệ phía máy chủ

- Kiểm tra ở máy khách chỉ để cải thiện trải nghiệm người dùng
- Kiểm tra ở máy chủ là yêu cầu bảo mật

Ví dụ thực tế:

Tôi từng thấy một ứng dụng kiểm tra giá sản phẩm ở máy khách. Kẻ tấn công sửa JavaScript, đổi giá từ \$100 thành \$1, rồi gửi biểu mẫu.

Máy chủ (Server) tin tưởng máy khách → Cơ sở dữ liệu lưu đơn hàng với giá \$1 → Công ty mất tiền.

Bài học: Máy khách là tầng hiển thị (*presentation layer*), không phải tầng nghiệp vụ (*business logic layer*).

Thành phần 2: Bộ Cân Bằng Tải (*Load Balancer*)

Vai trò: Phân phối lưu lượng (*traffic*) đều giữa nhiều máy chủ

Khi nào cần?

- Khi 1 máy chủ không đủ sức xử lý lưu lượng
- Khi cần tính sẵn sàng cao (*high availability*) (1 máy chủ chết, vẫn còn máy chủ khác)

Khi nào KHÔNG cần?

- Công ty khởi nghiệp với dưới 1000 người dùng
- Công cụ nội bộ với ít lưu lượng
- Bản thử nghiệm/MVP

Các đánh đổi (*trade-offs*):

Được:

- Phân phối tải đồng đều
- Không có điểm lỗi duy nhất
- Triển khai cuốn chiếu (cập nhật từng máy chủ)

Mất:

- Thêm 1 tầng phức tạp
- Bản thân bộ cân bằng tải có thể là điểm lỗi duy nhất (nếu không có dự phòng)
- Độ trễ mạng (+10-50ms)

Các thuật toán phổ biến:

Round Robin (xoay vòng):

Yêu cầu 1 → Máy chủ A
Yêu cầu 2 → Máy chủ B
Yêu cầu 3 → Máy chủ C
Yêu cầu 4 → Máy chủ A (lặp lại)

Đơn giản, phân phối đều. Nhưng không quan tâm máy chủ nào đang quá tải.

Least Connections (ít kết nối nhất):

Máy chủ A: 50 kết nối đang hoạt động
Máy chủ B: 20 kết nối đang hoạt động
Máy chủ C: 35 kết nối đang hoạt động

→ Yêu cầu mới được chuyển đến Máy chủ B

Thông minh hơn, nhưng cần theo dõi trạng thái.

Weighted Round Robin (xoay vòng có trọng số):

Máy chủ A (16GB RAM): trọng số = 2
Máy chủ B (8GB RAM): trọng số = 1

→ A nhận 2 yêu cầu cho mỗi 1 yêu cầu đến B

Hữu ích khi các máy chủ có cấu hình khác nhau.

Góc nhìn cá nhân: Trong thực tế, tôi thấy Round Robin là đủ cho 90% trường hợp. Least Connections chỉ cần khi các yêu cầu có thời gian xử lý rất khác nhau.

Thành phần 3: Máy Chủ Ứng Dụng

Vai trò: Xử lý nghiệp vụ, xử lý yêu cầu, điều phối luồng dữ liệu (*data flow*)

Phi trạng thái (Stateless) hay Có trạng thái - Quyết định quan trọng:

Máy chủ phi trạng thái:

```
# Không lưu user session trong memory
def handle_request(request):
    user_id = jwt.decode(request.token) # Get from token
    user = db.get_user(user_id)         # Get from DB
    return process(user)

# Benefit: Any server can handle any request
# Can scale horizontally easily
```

Máy chủ có trạng thái:

```
# Lưu user session trong memory
sessions = {} # In-memory storage

def handle_login(user_id):
    sessions[user_id] = UserSession(user_id)

def handle_request(user_id):
    session = sessions[user_id] # Retrieve from memory
    return process(session)

# Problem: User phải hit same server
# Scaling is hard (need sticky sessions)
```

Quyết định đánh đổi:

Chọn phi trạng thái khi:

- Cần mở rộng theo chiều ngang
- Nhiều máy chủ đứng sau bộ cân bằng tải
- Môi trường đám mây (máy chủ có thể chết bất cứ lúc nào)

Chọn có trạng thái khi:

- Cần độ trễ cực thấp (không phải gọi CSDL)
- Kết nối WebSocket (cần kết nối duy trì liên tục)
- Quy mô nhỏ (1-2 máy chủ)

Tôi khuyên: Mặc định chọn phi trạng thái. Chỉ chọn có trạng thái khi có lý do thật sự thuyết phục.

Thành phần 4: Lớp Bộ Nhớ Đệm (*Cache Layer*)

Vai trò: Lưu dữ liệu hay được truy cập trong bộ nhớ để giảm tải (*load*) cho cơ sở dữ liệu

Khi nào dùng bộ nhớ đệm?

Dùng bộ nhớ đệm khi:

- Đọc >> Ghi (tỷ lệ 10:1 trở lên)
- Dữ liệu không thay đổi thường xuyên
- Truy vấn tốn kém (JOIN phức tạp, tổng hợp)
- Các endpoint có lưu lượng cao

Không dùng bộ nhớ đệm khi:

- Tải công việc chủ yếu là ghi
- Dữ liệu thay đổi liên tục (giá cổ phiếu thời gian thực)
- Dữ liệu cá nhân hoá theo từng người dùng (mỗi người mỗi khác)

Ví dụ thực tế: Trang sản phẩm thương mại điện tử

```
# Product info: Perfect for cache
product = cache.get(f"product:{id}")
if not product:
    product = db.query("SELECT * FROM products WHERE id = ?", id)
    cache.set(f"product:{id}", product, ttl=3600) # 1 hour

# Inventory count: DON'T cache (changes frequently)
stock = db.query("SELECT stock FROM inventory WHERE product_id = ?", id)

# User's cart: DON'T cache (personalized)
cart = db.query("SELECT * FROM carts WHERE user_id = ?", user_id)
```

Vô Hiệu Hoá Bộ Nhớ Đệm (Cache Invalidation) - Bài Toán Khó:

Phil Karlton từng nói: "Chỉ có hai thứ khó trong Khoa học Máy tính: vô hiệu hoá bộ nhớ đệm và đặt tên."

Ông ấy đúng. Vô hiệu hoá bộ nhớ đệm là một cơn ác mộng.

Vấn đề:

1. Người dùng cập nhật hồ sơ → Cơ sở dữ liệu được cập nhật
2. Bộ nhớ đệm vẫn giữ dữ liệu cũ
3. Người dùng tải lại trang → Thấy dữ liệu cũ (trúng bộ nhớ đệm)
4. Người dùng bối rối, gửi báo cáo lỗi

Giải pháp:

Phương án 1: TTL (Thời gian sống (Time To Live))

```
cache.set("user:123", user, ttl=300) # Expire after 5 minutes
```

Simple

Data stale for up to 5 minutes

Phương án 2: Chủ động vô hiệu hoá

```
def update_user(user_id, data):
    db.update("users", user_id, data)
    cache.delete(f"user:{user_id}") # Delete immediately
```

Always fresh

Must remember to invalidate everywhere

Phương án 3: Mẫu Cache-Aside (Khuyến dùng)

```
def get_user(user_id):
    user = cache.get(f"user:{user_id}")
    if user:
        return user

    user = db.query("SELECT * FROM users WHERE id = ?", user_id)
    cache.set(f"user:{user_id}", user, ttl=3600)
    return user

def update_user(user_id, data):
    db.update("users", user_id, data)
    cache.delete(f"user:{user_id}") # Invalidate
    # Next read will miss cache → Fetch fresh from DB → Cache again
```

Nguyên tắc cá nhân: Bắt đầu đơn giản với TTL. Chỉ thêm cơ chế chủ động vô hiệu hoá khi độ tươi mới của dữ liệu thực sự quan trọng.

Thành phần 5: Cơ Sở Dữ Liệu

Vai trò: Lưu trữ bền vững, nguồn dữ liệu gốc (*source of truth*)

Cơ sở dữ liệu là điểm ngẽn phổ biến nhất.

Tại sao? Vì đọc/ghi đĩa (I/O) chậm hơn bộ nhớ 100.000 lần.

Đọc bộ nhớ: 100 nano giây
Đọc đĩa: 10 mili giây

10ms / 100ns = chậm hơn 100.000 lần

Hàm ý:

1. Giảm thiểu số lần gọi cơ sở dữ liệu

- Dùng bộ nhớ đệm
- Gộp truy vấn theo lô
- Tải sớm (eager loading) để tránh vấn đề N+1

2. Tối ưu truy vấn

- Thêm chỉ mục (indexes)
- Tránh SELECT *
- Dùng EXPLAIN ANALYZE

3. Mở rộng cơ sở dữ liệu sau cùng

- Tối ưu ứng dụng trước
- Thêm bộ nhớ đệm thứ hai
- Chỉ mở rộng CSDL khi thật cần thiết (tốn kém)

Ví dụ thực tế:

Một nhóm phân nà: "Cơ sở dữ liệu quá chậm, cần nâng cấp máy chủ!"

Tôi kiểm tra nhật ký truy vấn:

```
-- Running 1000 times per second
SELECT * FROM users WHERE email = 'john@example.com';
-- Execution time: 500ms
```

Vấn đề? Không có chỉ mục trên cột email → Phải quét toàn bộ bảng.

Giải pháp:

```
CREATE INDEX idx_users_email ON users(email);
-- Execution time: 5ms (100x faster!)
```

Chi phí: \$0 và 30 giây thiết lập.

Thay vì nâng cấp máy chủ (\$500/tháng), sửa truy vấn (miễn phí). **Luôn tối ưu trước khi mở rộng.**

Các Mẫu Giao Tiếp (*Communication Patterns*): Đồng Bộ hay Bất Đồng Bộ

Đây là một trong những quyết định quan trọng nhất trong thiết kế hệ thống (*system design*).

Đồng Bộ (*Synchronous*) (Yêu cầu - Phản hồi)

Cách hoạt động:

```
Máy khách → [Yêu cầu HTTP] → Máy chủ
Máy khách ← [Chờ...] ←
Máy khách ← [Phản hồi HTTP] ← Máy chủ
```

Đặc điểm:

- Máy khách bị chặn và chờ phản hồi
- Nhận phản hồi ngay lập tức
- Liên kết chặt (*Tight coupling*) (máy khách phụ thuộc máy chủ)

Khi nào dùng:

- Người dùng cần kết quả ngay lập tức
- Quy trình đơn giản
- Các thao tác CRUD

Ví dụ:

```
# User login
def login(email, password):
    user = authenticate(email, password) # Must know now
    if user:
        token = generate_token(user)
        return {"token": token}          # Return immediately
    return {"error": "Invalid credentials"}
```

Bất Đồng Bộ (Asynchronous) (Hàng Đợi Thông Điệp (Message Queue))

Cách hoạt động:

```
Máy khách → [Yêu cầu HTTP] → Máy chủ → [Đưa vào hàng đợi] → Trả về ngay
                                     ↓
                               Worker nền → Xử lý công việc
```

Đặc điểm:

- Máy khách không phải chờ
- Được xử lý sau
- Liên kết lỏng (*Loose coupling*) (máy khách không quan tâm ai xử lý)

Khi nào dùng:

- Xử lý mất nhiều thời gian (> 5 giây)
- Người dùng không cần kết quả ngay
- Cần chịu được các đợt tăng đột biến lưu lượng (*traffic spikes*)

Ví dụ:

```
# User uploads video
def upload_video(video_file):
    video_id = save_to_s3(video_file)          # Quick (2s)
    queue.add({
        "job": "process_video",
        "video_id": video_id,
        "tasks": ["thumbnail", "transcode", "notify"]
    })
    return {"status": "processing", "id": video_id} # Return immediately

# Background worker (runs separately)
def process_video(video_id):
    generate_thumbnail(video_id)    # 5s
    transcode_720p(video_id)       # 30s
    transcode_1080p(video_id)      # 60s
    send_notification(video_id)    # 1s
```

Trải nghiệm người dùng:

- Tải lên → Nhận phản hồi sau 2 giây (tốt)
- Video sẵn sàng → Nhận thông báo sau 1-2 phút (chấp nhận được)

Nếu làm đồng bộ → Người dùng phải chờ 98 giây → Hết thời gian chờ (Timeout) → Trải nghiệm tệ.

Ma Trận Đánh Đổi

Đồng bộ:
Logic đơn giản
Phản hồi ngay lập tức
Dễ gỡ lỗi
Trải nghiệm kém với thao tác chậm
Nguy cơ hết thời gian chờ
Không chịu được lưu lượng tăng đột biến

Bất đồng bộ:
Trải nghiệm tốt hơn (phản hồi nhanh)
Chịu được tăng đột biến (hàng đợi làm vùng đệm)
Có sẵn cơ chế thử lại
Phức tạp (cần hạ tầng hàng đợi)
Nhất quán cuối cùng
Khó gỡ lỗi hơn (cần truy vết phân tán)

Khung ra quyết định:

```
if user_needs_result_immediately:
    use_synchronous()
elif process_time > 5_seconds:
    use_asynchronous()
elif traffic_spiky: # Black Friday, viral post
    use_asynchronous()
else:
    use_synchronous() # Simpler
```

Luồng Dữ Liệu & Phát Hiện Điểm Nghẽn

Nguyên tắc vàng: Hệ thống chỉ nhanh bằng thành phần chậm nhất của nó.

Đo Lường Luồng Dữ Liệu

Ví dụ luồng: Người dùng tải trang chủ

Bước 1: Tra cứu DNS → 20ms
Bước 2: Bắt tay TCP → 50ms
Bước 3: Thương lượng TLS → 100ms
Bước 4: Gửi yêu cầu HTTP → 10ms
Bước 5: Máy chủ xử lý → 30ms
Bước 6: Truy vấn CSDL → 500ms ← ĐIỂM NGHẼN
Bước 7: Máy chủ dựng HTML → 20ms
Bước 8: Phản hồi về máy khách → 10ms
Bước 9: Trình duyệt phân tích/hiển thị → 200ms

Tổng: 940ms

Điểm nghẽn: Cơ sở dữ liệu (500ms = 53% tổng)

Kế hoạch hành động:

1. Tối ưu truy vấn cơ sở dữ liệu (thêm chỉ mục)
2. Thêm lớp bộ nhớ đệm
3. Đo lại

Sau khi tối ưu:

Bước 6: Trúng bộ nhớ đệm → 5ms

Tổng: 445ms (nhanh hơn 52%!)

Bài học: Đừng tối ưu bừa. Đo lường → Tìm điểm nghẽn → Tối ưu điểm nghẽn.

Các Điểm Nghẽn Phổ Biến & Giải Pháp

Điểm nghẽn 1: Cơ sở dữ liệu

Triệu chứng: Độ trễ truy vấn cao, CPU/IO chạm ngưỡng tối đa

Giải pháp:

- Thêm chỉ mục
- Tối ưu truy vấn
- Thêm bản sao chỉ đọc (read replica)
- Triển khai bộ nhớ đệm

Điểm nghẽn 2: Mạng

Triệu chứng: Độ trễ cao, hết thời gian chờ

Giải pháp:

- CDN cho tài nguyên tĩnh
- Giảm kích thước dữ liệu truyền (nén)
- Ít yêu cầu HTTP hơn (gộp tệp)

Điểm nghề 3: Logic ứng dụng

Triệu chứng: CPU sử dụng cao, phản hồi chậm

Giải pháp:

- Phân tích hiệu năng mã (profiling)
- Tối ưu thuật toán ($O(n^2) \rightarrow O(n \log n)$)
- Thêm bộ nhớ đệm
- Mở rộng theo chiều ngang

Bài Tập Thực Hành: Phân Tích Một Hệ Thống Thật

Hãy phân tích luồng đăng bài của Instagram.

Hành động của người dùng: Đăng một bức ảnh

Luồng hệ thống:

1. Ứng dụng di động → Tải ảnh lên → CDN
2. CDN → Trả về URL
3. Ứng dụng → API: Tạo bài đăng (URL, chú thích)
4. API → Lưu vào cơ sở dữ liệu (bảng posts)
5. API → Đưa vào hàng đợi: "fanout_post"
6. API → Trả về thành công cho người dùng
7. Worker nền → Lấy danh sách người theo dõi
8. Worker → Ghi bài đăng vào bảng tin của từng người theo dõi (bộ nhớ đệm)
9. Người theo dõi → Mở ứng dụng → Thấy bài đăng mới

Phân tích:

Các thành phần:

- Ứng dụng di động (máy khách)
- CDN (lưu trữ ảnh)
- Máy chủ API (phi trạng thái)
- Cơ sở dữ liệu (lưu trữ bền vững)
- Hàng đợi thông điệp (xử lý bất đồng bộ)

- Các tiến trình xử lý nền (*background workers*)
- Redis (bộ nhớ đệm cho bảng tin (*feed*))

Giao tiếp:

- Bước 1-6: Đồng bộ (người dùng cần xác nhận ngay)
- Bước 7-9: Bất đồng bộ (người theo dõi không cần cập nhật tức thì)

Điểm nghẽn:

- Nếu là người nổi tiếng với 10 triệu người theo dõi → Phải phân phát (*fanout*) tới 10 triệu bảng tin
- Giải pháp: Không phân phát cho người nổi tiếng, tải theo yêu cầu khi cần

Các đánh đổi:

- Người dùng bình thường: Đọc nhanh (bảng tin tạo sẵn), ghi chậm
- Người nổi tiếng: Ghi nhanh (không phân phát), đọc chậm hơn (truy vấn khi cần)

Mô Hình Tư Duy (*Mental Model*): Mọi Thứ Đều Là Một Hệ Thống

Áp dụng tư duy thành phần vào mọi thứ.

Quán cà phê:

- Thành phần: Thu ngân, Pha chế, Thiết bị, Kho nguyên liệu
- Giao tiếp: Phiếu gọi món (bất đồng bộ!)
- Điểm nghẽn: 1 người pha chế vs 10 khách hàng → Hàng chờ dài ra

Giải pháp (theo kiểu thiết kế hệ thống):

- Thêm người pha chế (mở rộng theo chiều ngang (*horizontal scaling*))
- Máy pha espresso tốt hơn (mở rộng theo chiều dọc (*vertical scaling*))
- Ki-ốt tự phục vụ (giảm tải cho thu ngân)

Nhà kho:

- Thành phần: Nhận hàng, Lưu kho, Lấy hàng, Giao hàng

- Giao tiếp: Hệ thống đơn hàng
- Điểm nghẽn: Lấy hàng thủ công → Chậm

Giải pháp:

- Robot lấy hàng (tự động hoá)
- Hệ thống kho tốt hơn (tối ưu lộ trình)

Khi bạn nhìn thế giới qua lăng kính hệ thống, bạn bắt đầu thấy những mẫu giống nhau ở khắp nơi.

Những Điểm Cốt Lõi

Mọi hệ thống đều có 3 yếu tố:

1. **Thành phần** - Những khối xây dựng thực hiện công việc
2. **Giao tiếp** - Cách các thành phần tương tác
3. **Ràng buộc** - Giới hạn về tài nguyên, độ trễ, thông lượng (*throughput*)

Để hiểu một hệ thống:

1. Xác định các thành phần
2. Vẽ luồng dữ liệu
3. Đo từng bước
4. Tìm điểm nghẽn
5. Tối ưu điểm nghẽn (không phải mọi thứ)

Hướng dẫn chọn đồng bộ hay bất đồng bộ:

- Đồng bộ: Người dùng cần kết quả ngay, quy trình đơn giản
- Bất đồng bộ: Xử lý lâu, lưu lượng tăng đột biến, cần tách rời các thành phần

Hướng dẫn dùng bộ nhớ đệm:

- Nên dùng: Đọc nhiều, truy vấn tốn kém, dữ liệu tĩnh
- Không nên dùng: Ghi nhiều, dữ liệu cá nhân hoá, thời gian thực

Tư duy thành phần không chỉ dành cho phần mềm. Nó là mô hình tư duy để phân tích bất kỳ hệ thống phức tạp nào.

Từ bây giờ, mỗi khi bạn nhìn một ứng dụng, hãy tự hỏi:

"Có những thành phần nào? Chúng giao tiếp thế nào? Điểm nghẽn ở đâu?"

Hãy luyện tập mô hình tư duy này. Nó sẽ thay đổi cách bạn nhìn nhận các hệ thống.

Tư Duy Theo Ràng Buộc (*Thinking in Constraints*): Kỹ Thuật Là Quản Lý Giới Hạn

Tôi còn nhớ lần đầu tiên thiết kế một hệ thống "cho 1 triệu người dùng".

Tôi vẽ một sơ đồ kiến trúc thật đẹp: vi dịch vụ (*microservices*), Kafka, Kubernetes, bộ nhớ đệm phân tán (*distributed cache*), tự động mở rộng (*auto-scaling*)...

Một kiến trúc sư lâu năm (*senior architect*) nhìn vào, hỏi một câu đơn giản: "Em có ngân sách bao nhiêu?"

Tôi: "Ngân sách ạ? Em chưa tính..."

Anh ấy: "Kiến trúc này tốn \$15.000/tháng. Ngân sách của em là \$2.000/tháng. Em nghĩ sao?"

Đó là lúc tôi học được bài học quan trọng nhất: Thiết kế không phải là vẽ sơ đồ đẹp. Thiết kế là làm việc với các ràng buộc (*constraints*).

Tại Sao Ràng Buộc Là Điểm Bắt Đầu?

Đối Chiếu Với Thực Tế

Lý thuyết: "Thiết kế hệ thống chịu được 1 triệu người dùng!"

Thực tế:

- Ngân sách: \$2.000/tháng (không phải vô hạn)
- Nhóm: 2 lập trình viên (không phải 50)
- Thời hạn: 2 tháng (không phải 2 năm)
- Hạ tầng sẵn có: PostgreSQL đã được cài đặt
- Chuyên môn: Nhóm chỉ biết Python và SQL

→ Kiến trúc hoàn hảo không tồn tại

→ Chỉ có kiến trúc phù hợp với các ràng buộc

Điểm mấu chốt: Ràng buộc không phải là kẻ thù. Ràng buộc là **các tham số thiết kế**.

Khung Tư Duy (Framework): Hệ Thống = Ràng Buộc + Đánh Đổi (Trade-offs)



Quy trình thiết kế hệ thống: Ràng buộc định hình quyết định, đánh đổi quyết định giải pháp

Hiểu ràng buộc → **Ra quyết định đúng** → **Tránh thiết kế thừa** (*over-engineering*)
hoặc thiết kế thiếu (*under-engineering*)

Ràng buộc 1: Giới Hạn Tính Toán

Thực tế: CPU không vô hạn.

Những Con Số Bạn Phải Biết

Năng lực của một lõi CPU:

- Máy chủ web thông thường: ~1.000 yêu cầu/giây
- Truy vấn cơ sở dữ liệu: ~10.000 truy vấn đơn giản/giây
- Tuần tự hoá JSON: ~50.000 đối tượng/giây
- Tính toán hàm băm: ~1.000.000 lần băm/giây

Các con số này có thể dao động, nhưng bậc độ lớn mới là điều quan trọng!

Kịch bản ví dụ:

Yêu cầu: Xử lý 10.000 yêu cầu/giây

Phương án 1: Một máy chủ cực mạnh (32 lõi)

- Năng lực lý thuyết: 32.000 req/s
- Chi phí: \$800/tháng
- Rủi ro: Điểm lỗi duy nhất

Phương án 2: 10 máy chủ tầm trung (mỗi máy 4 lõi)

- Năng lực: $10 \times 4.000 = 40.000$ req/s
- Chi phí: $10 \times \$200 = \2.000 /tháng
- Lợi ích: Có dự phòng, dễ mở rộng hơn

Phương án 3: Bắt đầu với 2 máy chủ, mở rộng sau

- Năng lực hiện tại: $2 \times 4.000 = 8.000$ req/s
- Chi phí: \$400/tháng
- Đối chiếu thực tế: Bạn có thực sự có 10 nghìn req/s không?

Chọn phương án nào?

→ Tùy vào các ràng buộc khác (ngân sách, tốc độ tăng trưởng, mức chịu rủi ro)

Ví Dụ Về Ràng Buộc Tính Toán

Trường hợp 1: Xử lý ảnh

Nhiệm vụ: Thay đổi kích thước 1 triệu ảnh

Một máy chủ: 100 ảnh/giây

Thời gian: $1 \text{ triệu} / 100 = 10.000$ giây = 2,8 giờ

Ràng buộc: Phải xong trong 30 phút

Tính toán: $1 \text{ triệu} / 1.800 \text{ giây} =$ cần 556 ảnh/giây

Giải pháp: 6 máy chủ ($6 \times 100 = 600$ ảnh/giây)

Phương án khác: Tối ưu thuật toán

- Xử lý song song: 400 ảnh/giây trên cùng phần cứng
- Chỉ cần 2 máy chủ thay vì 6
- Đánh đổi: Thời gian phát triển và chi phí hạ tầng

Trường hợp 2: Phân tích dữ liệu thời gian thực

Yêu cầu: Xử lý 100 nghìn sự kiện/giây

Đơn luồng: Tối đa 10 nghìn sự kiện/giây

Ràng buộc: Không chịu nổi tải

Các phương án:

1. Mở rộng theo chiều dọc (máy lớn hơn)
 - Máy chủ 16 lõi → 160 nghìn sự kiện/giây
 - Chi phí: \$1.200/tháng
2. Mở rộng theo chiều ngang (thêm máy)
 - 10 máy chủ 2 lõi → 200 nghìn sự kiện/giây
 - Chi phí: $10 \times \$100 = \1.000 /tháng
3. Tối ưu mã + dùng hàng đợi làm vùng đệm
 - Tối ưu lên 50 nghìn sự kiện/giây mỗi lõi
 - Máy chủ 2 lõi xử lý được 100 nghìn
 - Chi phí: \$200/tháng
 - Đánh đổi: Thời gian phát triển

Quyết định dựa trên ràng buộc:

- Ngân sách eo hẹp → Phương án 3
- Thời gian gấp → Phương án 1 hoặc 2
- Dài hạn → Phương án 3 rồi mở rộng theo Phương án 2

Ràng buộc 2: Giới Hạn Bộ Nhớ

Thực tế: RAM không miễn phí và có giới hạn vật lý.

Tính Toán Bộ Nhớ

Yêu cầu thông thường:

Mỗi phiên người dùng: ~1KB

1 triệu người dùng: 1GB RAM

Mỗi đối tượng lưu đệm: ~10KB

100 nghìn đối tượng lưu đệm: 1GB RAM

Mỗi kết nối cơ sở dữ liệu: ~10MB

100 kết nối: 1GB RAM

Một máy chủ 16GB:

- Hệ điều hành chiếm: ~2GB
 - Ứng dụng: ~2GB
 - Kết nối cơ sở dữ liệu (100): ~16GB
 - Bộ nhớ đệm: còn ~10GB
 - Phiên người dùng (10 triệu): Cần 10GB
- Vừa khít, hoặc phải nén/loại bỏ bớt

Ví dụ: Chiến lược lưu đệm (Caching strategy)

Thương mại điện tử: 1 triệu sản phẩm

Lưu đệm toàn bộ:

- Dữ liệu sản phẩm: mỗi cái 1KB
- Tổng: 1GB RAM
- Chi phí: Vừa vặn trong một instance Redis (\$50/tháng)
- Lợi ích: Mọi sản phẩm đều được lưu đệm

Lưu đệm một phần (sản phẩm "nóng"):

- 10% sản phẩm = 90% lưu lượng
- Bộ nhớ đệm: 100 nghìn sản phẩm = 100MB
- Chi phí: Nằm sẵn trong RAM của máy chủ ứng dụng
- Lợi ích: Miễn phí, đủ tốt

Chọn phương án nào?

- Kiểm tra ràng buộc: Ngân sách? Bộ nhớ còn trống?
- Nếu ngân sách ổn, lưu đệm toàn bộ = logic đơn giản hơn
- Nếu ngân sách eo hẹp, lưu đệm một phần = đủ dùng

Các Quyết Định Về Ràng Buộc Bộ Nhớ

Trường hợp: Bộ nhớ đệm dòng thời gian của người dùng

Ứng dụng mạng xã hội: 10 triệu người dùng

Phương án 1: Lưu đệm toàn bộ dòng thời gian

- 10 triệu người dùng $\times$ 100 bài $\times$ 1KB = 1TB RAM
- Chi phí: \$10.000/tháng (cụm Redis)
- Lợi ích: Dòng thời gian tải tức thì

Phương án 2: Chỉ lưu đệm người dùng đang hoạt động

- 1 triệu người dùng hoạt động $\times$ 100 bài $\times$ 1KB = 100GB RAM
- Chi phí: \$1.000/tháng
- Lợi ích: Tỷ lệ trùng bộ nhớ đệm 90% (đủ tốt)

Phương án 3: Lưu đệm theo nhu cầu

- Bộ nhớ đệm LRU, giới hạn 10GB
- Chi phí: \$100/tháng
- Lợi ích: Bao phủ dữ liệu "nóng" nhất
- Đánh đổi: Có một số lần trượt bộ nhớ đệm

Kiểm tra ràng buộc:

- 10 triệu người dùng, nhưng chỉ 1 triệu hoạt động hằng ngày
- Ngân sách: tổng \$2.000/tháng
- Phương án 2 hoặc 3 là hợp lý
- Phương án 1 là thiết kế thừa

Ràng buộc 3: Giới Hạn Mạng

Thực tế: Mạng có giới hạn về độ trễ (*latency*) và băng thông (*bandwidth*).

Các Định Luật Vật Lý

Tốc độ ánh sáng: 300.000 km/s

Nhưng trong cáp quang: 200.000 km/s

New York $\leftrightarrow$ San Francisco: 4.000 km

Độ trễ tối thiểu theo lý thuyết: $4.000 / 200.000 = 20\text{ms}$

Thực tế: 40-70ms (chỉ phí định tuyến)

→ Không thể thắng được vật lý

→ Đây là ràng buộc nền tảng

Ngân sách độ trễ:

Mức chịu đựng của người dùng: 100-200ms cho một lần tải trang web

Phân bổ ngân sách:

- Tra cứu DNS: 20ms
- Bắt tay TCP: 20ms
- Bắt tay TLS: 40ms
- Yêu cầu/Phản hồi: 20ms
- Máy chủ xử lý: ???

Đã dùng: 100ms

Còn lại cho máy chủ: 0-100ms

Ràng buộc: Máy chủ phải phản hồi trong tối đa 100ms

Ví Dụ Về Ràng Buộc Mạng

Trường hợp 1: Người dùng toàn cầu

Ứng dụng có người dùng khắp thế giới

Không có CDN:

- Người dùng Mỹ → Máy chủ Singapore: độ trễ 200ms
- Tài nguyên tĩnh (ảnh, JS, CSS): 5MB
- Thời gian tải: $200\text{ms} + (5\text{MB} / 10\text{Mbps}) = 200\text{ms} + 4\text{s} = 4,2\text{s}$
- Trải nghiệm người dùng: TỆ HẠI

Có CDN:

- Người dùng Mỹ → Điểm biên CDN ở Mỹ: độ trễ 20ms
- Tài nguyên tĩnh được lưu đệm tại điểm biên
- Thời gian tải: $20\text{ms} + (5\text{MB} / 50\text{Mbps}) = 20\text{ms} + 0,8\text{s} = 0,82\text{s}$
- Trải nghiệm người dùng: TỐT

Đánh đổi chi phí:

- Không có CDN: \$0
- Có CDN: \$50/tháng
- Quyết định: CDN đáng tiền (trải nghiệm người dùng là then chốt)

Trường hợp 2: Vị trí cơ sở dữ liệu

Máy chủ ứng dụng ở Mỹ, cơ sở dữ liệu ở châu Âu

Mỗi truy vấn:

- Mỹ → châu Âu: độ trễ 100ms
- Thực thi truy vấn: 10ms
- Châu Âu → Mỹ: độ trễ 100ms
- Tổng: 210ms mỗi truy vấn

Trang cần 10 truy vấn:

- Tổng: $2 \cdot 100\text{ms} = 2,1$ giây
- Không thể chấp nhận!

Giải pháp:

1. Chuyển cơ sở dữ liệu về Mỹ
 - Độ trễ: 10ms mỗi truy vấn
 - Tổng: 100ms cho 10 truy vấn
 - Đánh đổi: Vương vấn đề chủ quyền dữ liệu?
2. Nhân bản cơ sở dữ liệu (bản sao chỉ đọc đặt ở Mỹ)
 - Đọc: 10ms
 - Ghi: Vẫn mất 100ms tới máy chủ chính
 - Đánh đổi: Nhất quán cuối cùng
3. Lớp bộ nhớ đệm
 - Trúng bộ nhớ đệm: 1ms
 - Trượt bộ nhớ đệm: 210ms
 - Đánh đổi: Có thể gặp dữ liệu cũ

Quyết định dựa trên ràng buộc:

- Yêu cầu độ trễ: $< 200\text{ms}$ → Cần giải pháp
- Dữ liệu phải ở lại châu Âu → Phương án 2 hoặc 3
- Cần nhất quán mạnh → Phương án 2

Ràng buộc 4: Giới Hạn Cơ Sở Dữ Liệu

Thực tế: Cơ sở dữ liệu có giới hạn về thông lượng (throughput) và dung lượng lưu trữ.

Năng Lực Của Cơ Sở Dữ Liệu

Giới hạn thông thường của PostgreSQL:

Năng lực ghi:

- Một máy chủ: ~10.000 lượt ghi/giây
- Với SSD: ~50.000 lượt ghi/giây
- Ghi theo lô: ~100.000 lượt ghi/giây

Năng lực đọc:

- Truy vấn đơn giản: ~100.000 lượt đọc/giây
- Truy vấn phức tạp: ~1.000 lượt đọc/giây
- Có chỉ mục: cải thiện gấp 10 lần

Lưu trữ:

- Giới hạn thực tế: ~10TB mỗi máy chủ
- Vượt mức đó: Cân nhắc phân mảnh dữ liệu (sharding)

Kết nối:

- Số kết nối tối đa: ~500
- Thực tế: ~200 kết nối hoạt động

Ví dụ: Ứng dụng ghi nhiều

Hệ thống ghi log: 50.000 lượt ghi/giây

Một PostgreSQL duy nhất:

- Tối đa: 10.000 lượt ghi/giây
- Ràng buộc: Không chịu nổi tải

Các phương án:

1. Ghi theo lô

- Gom 100 lượt ghi, chèn cùng lúc
- Hiệu quả: 100.000 lượt ghi/giây
- Đánh đổi: Trễ một chút (chấp nhận được với log)

2. Dùng CSDL tối ưu cho ghi (Cassandra)

- Năng lực: 1.000.000 lượt ghi/giây
- Đánh đổi: Mô hình truy vấn khác, nhóm phải học thêm

3. Hàng đợi + worker chạy nền

- Nhận lượt ghi vào hàng đợi (tức thì)
- Worker ghi vào CSDL (tốc độ có kiểm soát)
- Đánh đổi: Dữ liệu được lưu bền vững sau một khoảng trễ

Kiểm tra ràng buộc:

- Nhóm thành thạo PostgreSQL
 - Log có thể trễ 1-2 giây
- Phương án 1 (ghi theo lô) là phù hợp nhất
- Đơn giản, tận dụng kiến thức sẵn có, đáp ứng yêu cầu

Ràng buộc 5: Giới Hạn Con Người (Quy Mô Nhóm)

Ràng buộc hay bị bỏ qua nhất: Năng lực của chính nhóm bạn.

Thực Tế Về Quy Mô Nhóm

Quy mô nhóm: 2 lập trình viên

Có thể duy trì:

- Hệ thống nguyên khối:  Dễ dàng
- 3-5 vi dịch vụ:  Khó khăn
- Hơn 10 vi dịch vụ:  Không thể
- Kubernetes:  Quá phức tạp
- Triển khai đơn giản:  Thiết yếu

Tại sao?

- Mỗi dịch vụ cần: triển khai, giám sát, gỡ lỗi
- 10 dịch vụ = gấp 10 lần gánh nặng vận hành
- 2 lập trình viên không thể quản lý tốt 10 dịch vụ

Khung Tư Duy Về Ràng Buộc Nhóm

Nhóm 1-3 người:

- ✓ Hệ thống nguyên khối
- ✓ Bộ công nghệ đơn giản (những gì nhóm đã biết)
- ✓ Dịch vụ được quản lý sẵn (giảm việc vận hành)
- × Vi dịch vụ
- × Hạ tầng phức tạp
- × Công nghệ mới

Nhóm 5-10 người:

- ✓ Hệ thống nguyên khối chia mô-đun
- ✓ Vi dịch vụ đơn giản (2-3 dịch vụ)
- ✓ Kết hợp công nghệ mới + quen thuộc
-  Vi dịch vụ toàn phần (rủi ro)

Nhóm từ 20 người trở lên:

- ✓ Vi dịch vụ
- ✓ Nhiều bộ công nghệ
- ✓ Có nhóm vận hành riêng
- ✓ Hạ tầng phức tạp

Ví dụ thực tế:

Công ty khởi nghiệp: 3 lập trình viên

Phương án A: Kiến trúc vi dịch vụ

- 8 dịch vụ
- Triển khai trên Kubernetes
- Service mesh
- Truy vết phân tán

Thời gian phát triển: 6 tháng

Bảo trì: 2 lập trình viên toàn thời gian chỉ để vận hành

Phát triển tính năng: CHẬM

Phương án B: Hệ thống nguyên khối có cấu trúc tốt

- Một kho mã nguồn, các mô-đun rõ ràng
- Triển khai trên Heroku
- Giám sát đơn giản

Thời gian phát triển: 6 tuần

Bảo trì: 0,5 lập trình viên cho vận hành

Phát triển tính năng: NHANH

Thực tế ràng buộc:

- 3 lập trình viên không thể duy trì 8 dịch vụ
 - Tốc độ ra thị trường là sống còn với công ty khởi nghiệp
- Phương án B là lựa chọn khả thi duy nhất

Danh Sách Kiểm Tra Ràng Buộc

Trước khi thiết kế bất kỳ hệ thống nào, hãy hỏi:

Ràng buộc kỹ thuật:

- ☐ Cần năng lực CPU bao nhiêu? (yêu cầu/giây)
- ☐ Bộ nhớ còn bao nhiêu? (kích thước bộ nhớ đệm, lưu phiên)
- ☐ Độ trễ mạng có chấp nhận được không? (vị trí người dùng, vị trí máy chủ)
- ☐ Thông lượng cơ sở dữ liệu có đủ không? (lượt ghi/đọc mỗi giây)
- ☐ Dung lượng lưu trữ? (kích thước dữ liệu hiện tại và 2 năm sau)

Ràng buộc tài nguyên:

- ☐ Ngân sách? (\$X/tháng cho hạ tầng)
- ☐ Thời hạn? (X tuần để ra mắt)
- ☐ Cái giá của sự cố? (có chịu được ngừng hoạt động không?)

Ràng buộc con người:

- ☐ Quy mô nhóm? (X lập trình viên)
- ☐ Chuyên môn của nhóm? (biết Y, cần học Z?)
- ☐ Năng lực vận hành? (ai duy trì hệ thống?)
- ☐ Bánh nặng trực sự cố? (có giám sát 24/7 được không?)

Ràng buộc tăng trưởng:

- ☐ Quy mô hiện tại? (X người dùng, Y yêu cầu/ngày)
- ☐ Tăng trưởng dự kiến? (gấp 2 lần/năm hay 10 lần/năm?)
- ☐ Tải đỉnh so với trung bình? (mẫu hình lưu lượng?)

Quy Trình Thiết Kế Dựa Trên Ràng Buộc

Khung tư duy từng bước:

1. Xác định TẤT CẢ ràng buộc
 - Giới hạn kỹ thuật
 - Giới hạn tài nguyên (ngân sách, thời gian)
 - Giới hạn con người (nhóm, chuyên môn)
2. Xếp hạng ưu tiên các ràng buộc
 - Đầu là giới hạn cứng? (không được vi phạm)
 - Đầu là giới hạn mềm? (có thể thương lượng)
3. Thiết kế trong phạm vi ràng buộc
 - Bắt đầu đơn giản
 - Chỉ thêm độ phức tạp khi ràng buộc đòi hỏi
4. Kiểm chứng với các ràng buộc
 - Giải pháp có vừa ngân sách không?
 - Nhóm có duy trì được không?
 - Có đáp ứng yêu cầu hiệu năng không?
5. Ghi lại các quyết định về ràng buộc
 - Tại sao chọn X thay vì Y?
 - Ràng buộc nào đã ảnh hưởng đến quyết định?

Ví dụ áp dụng:

Yêu cầu: Xây dựng bảng điều khiển phân tích dữ liệu

Các ràng buộc đã xác định:

- Ngân sách: \$500/tháng (giới hạn cứng)
- Nhóm: 1 lập trình viên (giới hạn cứng)
- Dữ liệu: 100GB (hiện tại), tăng 10GB/tháng
- Truy vấn: Tổng hợp phức tạp
- Người dùng: 50 người dùng nội bộ
- Thời hạn: 2 tuần để có MVP

Các quyết định dựa trên ràng buộc:

Chọn cơ sở dữ liệu:

- PostgreSQL ✓
 - Xử lý 100GB dễ dàng
 - Nhóm biết SQL
 - Truy vấn phức tạp: hỗ trợ sẵn
 - Chi phí: \$50/tháng cho bản được quản lý sẵn
- Cassandra ×
 - Quá mức cần thiết cho 100GB
 - Nhóm không biết
 - Tổng hợp phức tạp: khó
 - Đắt hơn

Triển khai:

- Heroku ✓
 - Đơn giản, được quản lý sẵn
 - \$200/tháng (trong ngân sách)
 - 1 lập trình viên quản lý được
- Kubernetes ×
 - Thiết lập phức tạp
 - Cần người vận hành chuyên trách
 - Vượt ngân sách nếu dùng bản quản lý sẵn

Bộ nhớ đệm:

- Bỏ qua cho MVP ✓
 - 50 người dùng = lưu lượng thấp
 - Có thể thêm sau nếu chậm
 - Tiết kiệm thời gian phát triển

Kết quả:

- Tổng chi phí: \$250/tháng (dưới ngân sách)
- Phát triển: 2 tuần (kịp thời hạn)
- Bảo trì: Tối thiểu (nhóm quản lý được)
- Hiệu năng: Đủ tốt cho quy mô hiện tại

Sai Lầm Phổ Biến: Phốt Lờ Ràng Buộc

Sai lầm 1: Thiết kế cho quy mô tưởng tượng

✗ Tệ:

"Biết đâu một ngày nào đó ta sẽ có 10 triệu người dùng, vậy dùng Kubernetes ngay bây giờ đi."

Thực tế:

- Hiện tại: 100 người dùng
- Thời hạn: 6 tháng để chứng minh sản phẩm phù hợp thị trường
- Ngân sách: \$2.000/tháng

✓ Tốt:

"Bắt đầu với Heroku. Nếu đạt 100 nghìn người dùng (thành công!), ta sẽ có doanh thu để chi trả việc chuyển sang Kubernetes. Đừng tối ưu cho những vấn đề ta chưa có."

Sai lầm 2: Phốt lờ ràng buộc về nhóm

✗ Tệ:

"Dùng Rust để đạt hiệu năng tối đa đi!"

Thực tế:

- Nhóm biết Python
- Yêu cầu hiệu năng: 1.000 req/s
- Python xử lý 1.000 req/s dễ dàng

✓ Tốt:

"Dùng Python. Nhóm làm việc hiệu quả ngay lập tức. Rust sẽ tốn 3 tháng để học. Ràng buộc hiệu năng không bị vi phạm."

Sai lầm 3: Mặc định ngân sách là vô hạn

✗ Tệ:

"Ta sẽ dùng Kubernetes quản lý sẵn, CSDL quản lý sẵn, bộ nhớ đệm quản lý sẵn, mọi thứ đều quản lý sẵn!"

Chi phí: \$8.000/tháng

Thực tế:

- Ngân sách: \$2.000/tháng
- Vi phạm ràng buộc cứng

✓ Tốt:

"Dùng Heroku (\$200), PostgreSQL quản lý sẵn (\$50), và bộ nhớ đệm ở tầng ứng dụng (miễn phí).

Tổng: \$250/tháng. Vừa ngân sách."

Những Điểm Cốt Lõi

Kỹ thuật = Quản lý ràng buộc

Không phải: "Kiến trúc nào là tốt nhất?"

Mà là: "Kiến trúc nào phù hợp với các ràng buộc của tôi?"

5 nhóm ràng buộc:

1. Tính toán: Giới hạn năng lực CPU
2. Bộ nhớ: Giới hạn RAM sẵn có
3. Mạng: Giới hạn độ trễ và băng thông
4. Cơ sở dữ liệu: Giới hạn thông lượng và lưu trữ
5. Con người: Giới hạn quy mô và chuyên môn của nhóm

Khung tư duy:

Thiết kế hệ thống = Ràng buộc + Đánh đổi

Ràng buộc → Xác định điều gì khả thi

Đánh đổi → Chọn giữa các khả năng

Vừa vặn → Giải pháp tôn trọng các ràng buộc

Quy trình thiết kế:

1. Xác định ràng buộc (tất cả!)
2. Xếp ưu tiên (giới hạn cứng hay mềm)
3. Thiết kế trong phạm vi ràng buộc
4. Kiểm chứng giải pháp có phù hợp
5. Ghi lại các quyết định

Hãy nhớ:

Kiến trúc hoàn hảo nhưng phớt lờ ràng buộc = Vô dụng
Kiến trúc đủ tốt và tôn trọng ràng buộc = Có thể ra mắt

Ràng buộc không phải kẻ thù
Ràng buộc là tham số thiết kế

Làm việc CÙNG ràng buộc, đừng CHỐNG LẠI chúng

Sự thay đổi tư duy cần có:

Từ: "Làm sao xây hệ thống hoàn hảo?"

Sang: "Làm sao xây đúng hệ thống với các ràng buộc của tôi?"

Từ: "Thực hành tốt nhất bảo X"

Sang: "Ràng buộc của tôi đòi hỏi Y"

Từ: "Chuẩn bị sẵn cho mọi thứ trong tương lai"

Sang: "Giải quyết ràng buộc hôm nay, thích ứng ngày mai"

Bạn đã học tư duy ưu tiên vấn đề (Bài 1) và tư duy đánh đổi (*trade-off thinking*) (Bài 2).

Giờ với tư duy ràng buộc (bài này), bạn đã có một mô hình tư duy (*mental model*) hoàn chỉnh:

Vấn đề → Ràng buộc → Đánh đổi → Giải pháp

Đây là cách các kiến trúc sư lâu năm thực sự suy nghĩ.

Scale Intuition: Xây Dựng Cảm Giác Về Numbers

Tôi còn nhớ lần đầu tiên được hỏi trong interview:

"Hệ thống của em cần handle bao nhiêu requests per second?"

Tôi: "Uhm... nhiều ạ?"

Interviewer: "Nhiều là bao nhiêu? 10? 1,000? 1,000,000?"

Tôi: "... Em không biết ạ."

Tôi fail câu hỏi đó.

Không phải vì không biết code. Mà vì **không có scale intuition**.

Senior architect ngồi bên cạnh sau đó nói: "Em biết không, difference giữa 1K users và 1M users không phải là 1000 lần. Nó là difference giữa một chiếc xe đạp và một chiếc Boeing 747."

Đó là lúc tôi bắt đầu học về scale.

Tại Sao Scale Intuition Quan Trọng?

The Problem: Numbers Are Abstract

Khi bạn nghe "1 million users", bạn nghĩ gì?

Hầu hết developers: "Ồ, nhiều quá!"

Nhưng "nhiều" không giúp bạn design system.

Bạn cần biết:

- 1 million users = bao nhiêu requests/second?
- Cần bao nhiêu servers?
- Bao nhiêu database capacity?
- Bao nhiêu storage?

- Bao nhiêu cost?

The Solution: Develop Intuition

Scale intuition = Ability to quickly estimate system requirements

Skill: "Ước chừng 1 million users cần 10 app servers, 2TB storage, cost ~\$3K/tháng"

Not: "1 million users... cần nhiều servers... uhm..."

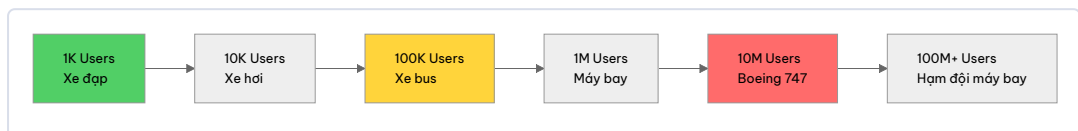
Có intuition → Design với confidence

Không có intuition → Design bằng guessing

Bài này sẽ dạy bạn build intuition đó.

The Scale Spectrum: 1K vs 1M vs 1B Users

Visualizing the Difference



Mỗi bậc scale là một thế giới hoàn toàn khác nhau

1,000 Users (Startup MVP)

Daily Active Users (DAU): ~500

Concurrent users: ~50

Requests per second: ~5-10

Infrastructure:

- 1 application server (\$50/month)
- 1 database server (\$50/month)
- No load balancer needed
- No caching needed

Total cost: ~\$100/month

Bottleneck: Probably none

Architecture: Simple monolith

Team: 1-2 developers can handle

Real example:

Local coffee shop app:

- 500 customers
- Each checks menu 2 times/day
- 1,000 requests/day
- $1,000 / 86,400 \text{ seconds} \approx 0.01 \text{ requests/second}$

Can run on Raspberry Pi! 😊

10,000 Users (Growing Startup)

Daily Active Users: ~5,000

Concurrent users: ~500

Requests per second: ~50-100

Infrastructure:

- 2-3 application servers (\$150/month)
- 1 database server (\$100/month)
- Load balancer (\$50/month)
- Redis cache optional (\$50/month)

Total cost: ~\$350/month

Bottleneck: Database queries (add indexes)

Architecture: Monolith + cache

Team: 2-5 developers

Key difference từ 1K:

- Cần load balancer (high availability)
- Database indexes matter
- Monitoring starts to be important

100,000 Users (Established Product)

Daily Active Users: ~50,000
Concurrent users: ~5,000
Requests per second: ~500-1,000

Infrastructure:

- 10 application servers (\$500/month)
- Database với read replicas (\$500/month)
- Load balancer (\$100/month)
- Redis cluster (\$300/month)
- CDN (\$100/month)

Total cost: ~\$1,500/month

Bottleneck: Database writes, cache invalidation

Architecture: Modular monolith hoặc simple microservices

Team: 10-20 developers

Key difference từ 10K:

- Database read replicas necessary
- Caching is critical
- CDN for static assets
- Auto-scaling starts to matter

1,000,000 Users (Large Scale)

Daily Active Users: ~500,000
Concurrent users: ~50,000
Requests per second: ~5,000-10,000

Infrastructure:

- 50+ application servers (\$2,500/month)
- Sharded database cluster (\$2,000/month)
- Multiple load balancers (\$300/month)
- Distributed cache (\$1,000/month)
- CDN (\$500/month)
- Message queues (\$200/month)

Total cost: ~\$6,500/month+

Bottleneck: Everything! Need distributed approach

Architecture: Microservices likely

Team: 50+ developers, dedicated ops team

Key difference từ 100K:

- Database sharding needed

- Distributed systems challenges
- Dedicated DevOps/SRE team
- Complex monitoring and alerting

The Pattern

Mỗi 10x scale:

- Cost tăng ~3-5x (not linear!)
- Complexity tăng significantly
- Team size tăng
- Architecture thay đổi fundamentally

1K → 10K: Add servers, basic optimization

10K → 100K: Add caching, read replicas

100K → 1M: Sharding, microservices, distributed systems

1M → 10M: Geographic distribution, edge computing

Back-of-Envelope Calculations

Core skill: Quickly estimate system requirements

Calculation Framework

1. Start với users
2. Estimate usage patterns
3. Calculate requests
4. Estimate storage
5. Calculate bandwidth

Exercise 1: E-commerce Website

Given:

- 1 million registered users
- 20% active daily (200K DAU)
- Each user views 10 product pages/day
- Each user makes 1 purchase/month

Calculate:

Requests per second:

Page views:

$200\text{K users} \times 10 \text{ pages/day} = 2\text{M page views/day}$

$2\text{M} / 86,400 \text{ seconds/day} \approx 23 \text{ requests/second}$

Peak traffic (assume 3x average):

$23 \times 3 = 69 \text{ requests/second}$

API calls (assume 5 API calls per page view):

$23 \times 5 = 115 \text{ requests/second}$

Peak: 345 requests/second

→ Need to handle ~350 req/s at peak

Storage:

Product catalog:

$100\text{K products} \times 10\text{KB each} = 1\text{GB}$

Product images:

$100\text{K products} \times 5 \text{ images} \times 500\text{KB} = 250\text{GB}$

User data:

$1\text{M users} \times 1\text{KB} = 1\text{GB}$

Orders (1 year):

$1\text{M users} \times 12 \text{ purchases/year} \times 5\text{KB} = 60\text{GB}$

Total: ~312GB

→ Single database can handle easily

→ Images need CDN/S3

Bandwidth:

Page views: 2M/day

Average page size: 2MB (HTML + images + JS)

Total bandwidth: $2\text{M} \times 2\text{MB} = 4\text{TB/day}$

With CDN cache (90% cache hit rate):

$4\text{TB} \times 0.1 = 400\text{GB/day from origin}$

= 16GB/hour from servers

= Manageable with CDN

Infrastructure estimate:

Application servers:

- 350 req/s peak
- Each server: ~100 req/s
- Need: 4 servers (with buffer)

Database:

- 312GB data (single server OK)
- Read-heavy (add 2 read replicas)

CDN: Essential (90% traffic)

Total cost: ~\$800/month

Exercise 2: Social Media App

Given:

- 10 million users
- 1 million daily active users
- Each user posts 2 times/day
- Each user views 100 posts/day

Calculate:

Write traffic:

Posts created:

$1\text{M users} \times 2 \text{ posts/day} = 2\text{M posts/day}$

$2\text{M} / 86,400 \approx 23 \text{ writes/second}$

Peak: 69 writes/second

→ Single database can handle

Read traffic:

Posts viewed:

$1\text{M users} \times 100 \text{ posts/day} = 100\text{M views/day}$

$100\text{M} / 86,400 \approx 1,157 \text{ reads/second}$

Peak: 3,471 reads/second

→ Need caching!

Storage growth:

Posts per day: 2M
Post size: 500 bytes (text + metadata)
Daily storage: $2M \times 500 \text{ bytes} = 16B/\text{day}$

Photos (50% of posts have 1 photo):
 $1M \text{ photos/day} \times 2MB = 2TB/\text{day}$

Annual growth:
Text: $16B \times 365 = 365GB$
Photos: $2TB \times 365 = 730TB$

- Need distributed storage (S3)
- Database: Shard after 1-2 years

Read:Write ratio:

Reads: 1,157/s
Writes: 23/s
Ratio: 50:1

- Heavy read optimization needed
- Aggressive caching strategy
- CDN for media

Requests Per Second (RPS) Intuition

What Different RPS Means

1-10 RPS:

- Small internal tool
- Single server sufficient
- No special optimization

10-100 RPS:

- Small web app
- 1-2 servers
- Basic caching helps

100-1,000 RPS:

- Medium web app
- 5-10 servers
- Caching essential
- Database optimization matters

1,000-10,000 RPS:

- Large web app
- 20-50 servers
- Distributed cache
- Database read replicas
- Load balancing critical

10,000-100,000 RPS:

- Very large scale
- 100+ servers
- Database sharding
- Geographic distribution
- CDN mandatory

100,000+ RPS:

- Massive scale (Google, Facebook level)
- Thousands of servers
- Custom infrastructure
- Edge computing
- Advanced optimization everywhere

Server Capacity Rule of Thumb

Typical web server:

- Simple queries: ~1,000 req/s
- Medium complexity: ~500 req/s
- Complex logic: ~100 req/s

Database:

- Simple reads: ~10,000 req/s
- Simple writes: ~5,000 req/s
- Complex queries: ~100 req/s
- Transactions: ~1,000 req/s

Cache (Redis):

- Reads: ~100,000 req/s per node
- Writes: ~50,000 req/s per node

Use these to estimate:

Need 5,000 req/s with medium complexity?

$5,000 / 500 = 10$ servers minimum

Add 50% buffer: 15 servers

Database can't keep up?

Use caching to reduce DB load by 90%

$5,000 \times 0.1 = 500$ req/s to DB → Single DB OK

Storage Growth Intuition

Storage Math

Common data sizes:

User record: ~1KB

- username, email, hashed password, metadata

Tweet/Post: ~500 bytes

- text content, user_id, timestamp, metadata

Photo (compressed): ~2MB

- JPEG, optimized for web

Video (1 min, 720p): ~50MB

- H.264 compression

Log entry: ~200 bytes

- timestamp, level, message

Growth Calculation Example

Scenario: Photo sharing app

Users: 1 million
Active daily: 200K
Photos uploaded per active user: 2

Daily uploads:
 $200K \times 2 = 400K$ photos/day

Daily storage:
 $400K \times 2MB = 800GB/day$

Monthly: $800GB \times 30 = 24TB/month$
Yearly: $24TB \times 12 = 288TB/year$

Cost (AWS S3):
\$0.023/GB/month
 $288TB = 288,000GB$
Cost: $288,000 \times \$0.023 = \$6,624/month$

→ Significant! Need compression, CDN, storage tiers

Optimization strategies:

Original calculation: 288TB/year

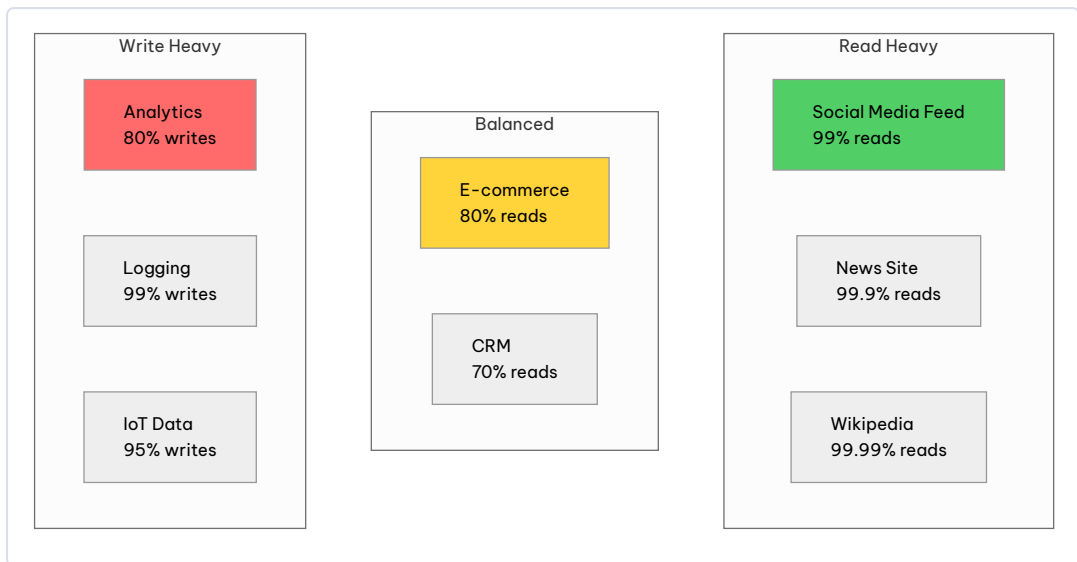
With optimization:

1. Aggressive compression: -30% = 200TB
2. Delete old/unused photos: -20% = 160TB
3. Use storage tiers (cold storage): -40% cost
4. Total: 160TB at \$0.014/GB = \$2,240/month

Savings: \$4,384/month (66% reduction!)

Read vs Write Heavy Patterns

Pattern Recognition



Khác pattern cần khác optimization strategy

Read-Heavy Systems (90%+ reads)

Examples: Social media, news, blogs, documentation

Characteristics:

- Many users viewing same content
- Content doesn't change often
- Cache hit rate very high

Optimization strategy:

- ✓ Aggressive caching (Redis, Memcached)
- ✓ CDN for static content
- ✓ Database read replicas
- ✓ Eventual consistency OK
- × Don't optimize writes (not bottleneck)

Infrastructure focus:

- Cache layer (most important)
- CDN (for media)
- Multiple read replicas

Example: Reddit

- 1 post → 10,000 views
- Cache post for 5 minutes
- 99.99% cache hit rate
- Database load minimal

Write-Heavy Systems (50%+ writes)

Examples: Analytics, logging, IoT sensors, financial trading

Characteristics:

- Constant data ingestion
- Reads less frequent
- Data often time-series

Optimization strategy:

- ✓ Write-optimized database (Cassandra, time-series DB)
- ✓ Async writes (queue buffering)
- ✓ Batch processing
- ✓ Sharding/partitioning
- × Caching less effective

Infrastructure focus:

- Write throughput (database)
- Message queues (buffer)
- Batch processing

Example: Monitoring system

- 10K servers × 100 metrics/minute
- 1M writes/minute = 16K/second
- Reads: Once/day for dashboards
- Optimize for write throughput

Balanced Systems (50-80% reads)

Examples: E-commerce, CRM, productivity tools

Characteristics:

- Mix of reading and writing
- Need to optimize both
- More complex trade-offs

Optimization strategy:

- ✓ Selective caching (hot data only)
- ✓ Database optimization (indexes)
- ✓ Read replicas for reports
- ✓ Write optimization for transactions

Infrastructure focus:

- Balanced approach
- Cache hot paths
- Optimize critical queries

Bottleneck Mindset

Core principle: System is only as fast as its slowest part

The Bottleneck Chain

Request flow:

```
Client → (Network 50ms)
      → Load Balancer (5ms)
      → App Server (20ms)
      → Database Query (500ms) ← BOTTLENECK
      → App Server (10ms)
      → Client (50ms)
```

Total: 635ms

Bottleneck: Database (79% of time)

Optimization priority:

1. Database (biggest impact)
2. Network (if can't fix DB)
3. App logic (minimal impact)

Don't waste time optimizing app server from 20ms → 10ms
when database takes 500ms!

Identifying Bottlenecks

Ask these questions:

1. What takes the most time?
 - Measure each component
 - Find the slowest
2. What has the highest load?
 - CPU usage
 - Memory usage
 - Network saturation
 - Disk I/O
3. What fails first under load?
 - Database connections maxed?
 - Server memory full?
 - Network bandwidth saturated?

The bottleneck is where you hit limits first.

Common Bottlenecks by Scale

1K users:

- Bottleneck: Usually none
- If any: Inefficient queries

10K users:

- Bottleneck: Database queries
- Fix: Add indexes, caching

100K users:

- Bottleneck: Database writes
- Fix: Read replicas, optimization

1M users:

- Bottleneck: Database capacity
- Fix: Sharding, distributed cache

10M+ users:

- Bottleneck: Everything
- Fix: Distributed architecture, CDN, edge computing

Practice Exercises

Exercise 1: Estimate YouTube-like Service

Given:

- 1 billion users
- 100 million daily active users
- Each user watches 10 videos/day (average 5 min each)
- 1% of users upload 1 video/day

Calculate:

1. Video views per second
2. Upload bandwidth needed
3. Storage growth per day
4. Approximate infrastructure cost

Try it yourself first!

Answer:

1. Video views per second:
 $100\text{M users} \times 10 \text{ videos/day} = 1\text{B views/day}$
 $1\text{B} / 86,400 \text{ seconds} \approx 11,574 \text{ views/second}$
2. Upload bandwidth:
 $1\text{M uploads/day (1\% of 100M)}$
Average video: $5 \text{ min} \times 10\text{MB/min} = 50\text{MB}$
 $50\text{MB} \times 1\text{M} = 50\text{TB/day upload bandwidth}$
 $50\text{TB} / 86,400\text{s} \approx 580 \text{ MB/second upload}$
3. Storage growth:
 $1\text{M videos/day} \times 50\text{MB} = 50\text{TB/day raw}$
Multiple resolutions (360p, 720p, 1080p): $\times 3$
 $= 150\text{TB/day}$
 $= 4.5\text{PB/month}$
4. Infrastructure cost (rough):
Storage: $4.5\text{PB} \times \$0.02/\text{GB} = \$90,000/\text{month}$
CDN: $1\text{B views} \times 50\text{MB} = 50\text{PB delivery}$
CDN cost: $\sim \$500,000/\text{month}$
Compute: $\sim \$200,000/\text{month}$
Total: $\sim \$800,000/\text{month minimum}$

Exercise 2: Estimate Messaging App

Given:

- 500 million users
- 200 million daily active
- Each user sends 50 messages/day
- Each user receives 50 messages/day

Calculate:

1. Messages per second
2. Storage per day
3. Database writes per second
4. Real-time delivery challenge

Your turn!

Answer:

1. Messages per second:
 $200\text{M users} \times 50 \text{ messages} = 10\text{B messages/day}$
 $10\text{B} / 86,400 \approx 115,740 \text{ messages/second}$
2. Storage per day:
 $10\text{B messages} \times 100 \text{ bytes} = 1\text{TB/day}$
(text messages are small)
3. Database writes:
 $115,740 \text{ writes/second}$
→ Need distributed database or write buffering
4. Real-time delivery:
 $115,740 \text{ messages/s to deliver}$
Each needs WebSocket push
→ Need efficient pub/sub system
→ Message queues essential

Building Your Intuition: Practice Guide

How to develop scale intuition:

Step 1: Memorize Key Numbers

Must know:

- 1 million seconds $\approx$ 12 days
- 1 billion seconds $\approx$ 32 years
- 1KB = 1,000 bytes
- 1MB = 1,000KB = 1 million bytes
- 1GB = 1,000MB = 1 billion bytes
- 1TB = 1,000GB

Typical server:

- 1,000 requests/second
- 16GB RAM
- 1TB storage

Typical costs:

- Server: \$50-500/month
- Database: \$50-1,000/month
- CDN: \$10-1,000/month
- Storage: \$0.02/GB/month

Step 2: Practice Daily Estimation

Every day, estimate:

Monday: "Instagram has 500M DAU. How many photos uploaded/day?"

Tuesday: "Twitter handles X tweets/second?"

Wednesday: "Netflix bandwidth per second?"

Thursday: "Google searches per second?"

Friday: "Facebook storage growth per day?"

Practice makes perfect!

Step 3: Verify Your Estimates

After estimating, research actual numbers:

- Instagram: 95M photos/day
- Twitter: 6,000 tweets/second
- Netflix: 200Gbps peak bandwidth
- Google: 40,000 searches/second

Compare với your estimates

Calibrate your intuition

Key Takeaways

Scale intuition = Quickly estimate system requirements

Critical skill for:

- System design interviews
- Architecture planning
- Capacity planning
- Cost estimation

The scale spectrum:

1K users: Bicycle (simple)

10K users: Car (add structure)

100K users: Bus (need optimization)

1M users: Airplane (distributed systems)

10M+ users: Fleet of airplanes (complex infrastructure)

Each 10x = fundamentally different architecture

Back-of-envelope calculations:

Framework:

1. Users → Activity → Requests
2. Requests → Server capacity
3. Data → Storage → Cost
4. Identify bottlenecks

Practice until automatic!

Read vs Write patterns:

Read-heavy (90%+):

- Caching is king
- CDN essential
- Read replicas

Write-heavy (50%+):

- Write optimization
- Queue buffering
- Specialized databases

Know your pattern → Know your strategy

Bottleneck mindset:

System speed = Slowest component speed

Always ask:

- What's the bottleneck?
- What hits limits first?
- What to optimize first?

Optimize bottleneck, not everything!

Building intuition:

1. Memorize key numbers
2. Practice daily estimates
3. Verify with real data
4. Calibrate and improve

After 100 estimates, you'll have strong intuition

Remember:

Intuition không đến từ theory
Intuition đến từ practice

Làm calculations mỗi ngày
Sau 1 tháng, bạn sẽ "cảm" được scale

System Thinking Framework: Nhìn Hệ Thống Như Một Tổng Thể

Tôi còn nhớ lần đầu tiên nhìn vào codebase của một hệ thống lớn.

Hàng trăm files. Chục microservices. Database, cache, queues, APIs...

Tôi hỏi senior: "Làm sao anh hiểu được hết những cái này?"

Anh ấy cười: "Em không cần hiểu từng dòng code. Em cần hiểu hệ thống như một **tổng thể**."

Tôi: "Nghĩa là sao ạ?"

Anh ấy vẽ một diagram đơn giản:

```
User → API → Cache → Database
           ↓
        Queue → Worker
```

"Đây là cả hệ thống. Data flow từ User, qua API, check Cache, query Database, async jobs vào Queue. Hiểu flow này quan trọng hơn hiểu từng function."

Đó là lần đầu tiên tôi học về System Thinking.

Tại Sao Phải Think in Systems?

The Problem: Thinking in Code

Hầu hết developers think như này:

```
# Code-level thinking
def get_user(user_id):
    user = cache.get(user_id)
    if not user:
        user = db.query("SELECT * FROM users WHERE id = ?", user_id)
        cache.set(user_id, user)
    return user
```

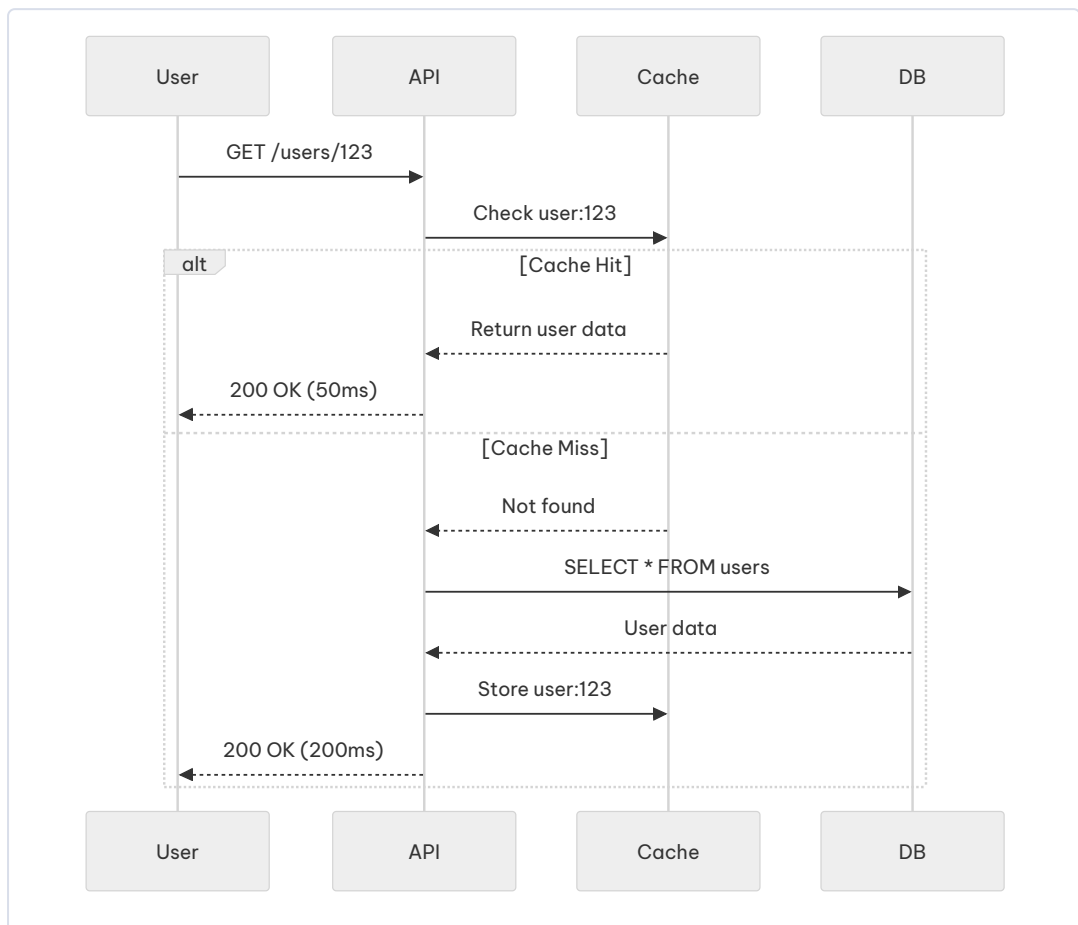
Bạn thấy: function, cache logic, database query.

Nhưng bạn không thấy:

- Request flow từ đâu đến?
- Response đi về đâu?
- Cache fail thì sao?
- Database slow thì sao?
- Scale như thế nào?

The Solution: System-Level View

System thinking nhìn như này:



System view: Thấy được flow, interactions, và failure scenarios

Thấy sự khác biệt chưa?

System thinking giúp bạn:

- Hiểu data flow
- Spot bottlenecks
- Design for failure
- Scale intelligently
- Debug production issues

The Universal System Pattern

Mọi system đều follow một pattern cơ bản:

Input → Processing → Storage → Output



Universal pattern: Mọi system đều nhận input, xử lý, lưu trữ, và trả output

Examples of This Pattern

Web Application:

Input: User clicks "Submit Order"
Processing: Validate, calculate total, check inventory
Storage: Save order to database
Output: Show confirmation page

Messaging System:

Input: User sends message
Processing: Encrypt, route to recipient
Storage: Store in message queue
Output: Deliver to recipient's device

Analytics Pipeline:

Input: Events from applications
Processing: Aggregate, transform data
Storage: Write to data warehouse
Output: Update dashboards

Video Platform:

Input: User uploads video
Processing: Encode to multiple formats
Storage: Save to object storage (S3)
Output: Video available for streaming

Key insight: Hiểu pattern này → Hiểu mọi system.

Core System Components

Mọi web system đều có 6 components cơ bản:

Component 1: Client (Input Layer)

Vai trò: Initiate requests, receive responses

Types:

- Web browser (JavaScript)
- Mobile app (iOS/Android)
- Desktop app
- Other services (API clients)

Characteristics:

- Không đáng tin cậy (user có thể modify)
- Không kiểm soát được (network, device)
- Stateful (có local state)

Key principle: NEVER trust client input

Example flow:

User action:

1. User clicks "Login"
2. Browser sends POST /login with credentials
3. Waits for response
4. Renders result

What can go wrong:

- User modifies request (change user_id)
- Network timeout
- Slow response
- Device crashes

System must handle all these!

Component 2: API (Processing Layer)

Vai trò: Handle requests, execute business logic, orchestrate data flow

Responsibilities:

- Validate input (client không tin cậy!)
- Execute business rules
- Coordinate between services
- Format response
- Handle errors

Characteristics:

- Stateless (nên là stateless!)
- Horizontally scalable
- Contains business logic

Technology examples:

- REST API (Express, FastAPI, Spring)
- GraphQL API
- gRPC services

Example:

```
# API Server handles request
@app.post("/orders")
def create_order(order_data):
    # 1. Validate (client không tin cậy)
    if not validate_order(order_data):
        return 400, {"error": "Invalid order"}

    # 2. Business logic
    total = calculate_total(order_data)

    # 3. Check inventory
    if not check_inventory(order_data.items):
        return 400, {"error": "Out of stock"}

    # 4. Save to database
    order = db.create_order(order_data, total)

    # 5. Async processing (queue)
    queue.publish("order.created", order.id)

    # 6. Return response
    return 201, {"order_id": order.id}
```

Component 3: Database (Storage Layer)

Vai trò: Persist data reliably

Types:

- Relational (PostgreSQL, MySQL)
 - Structured data, transactions
- NoSQL (MongoDB, Cassandra)
 - Flexible schema, horizontal scale
- Time-series (InfluxDB, TimescaleDB)
 - Metrics, logs, events

Characteristics:

- ACID properties (SQL) or eventual consistency (NoSQL)
- Slower than memory (disk I/O)
- Usually the bottleneck
- Data must survive crashes

Key principle: Source of truth

When to use what:

SQL (PostgreSQL):

- ✓ Need transactions
- ✓ Complex relationships
- ✓ Data integrity critical

Example: E-commerce, banking

NoSQL (MongoDB):

- ✓ Flexible schema
- ✓ Horizontal scaling
- ✓ Simple queries

Example: User profiles, content

Time-series (InfluxDB):

- ✓ Metrics/monitoring
- ✓ IoT sensor data
- ✓ Time-based queries

Example: Observability, analytics

Component 4: Cache (Performance Layer)

Vai trò: Store frequently accessed data in memory

Why cache?

- Memory: 100 nanoseconds
- SSD: 150 microseconds (1,500x slower)
- HDD: 10 milliseconds (100,000x slower)

→ Cache = Speed up reads dramatically

Common implementations:

- Redis (most popular)
- Memcached (simple key-value)
- Application-level (in-process)

Characteristics:

- Volatile (data can be lost)
- Limited capacity (expensive RAM)
- Stale data possible
- Cache invalidation is hard

Key principle: Cache is optimization, not source of truth

Cache patterns:

```

# Pattern 1: Cache-Aside (Lazy Loading)
def get_user(user_id):
    # Check cache first
    user = cache.get(f"user:{user_id}")
    if user:
        return user # Cache hit

    # Cache miss → Query database
    user = db.query("SELECT * FROM users WHERE id = ?", user_id)

    # Store in cache for next time
    cache.set(f"user:{user_id}", user, ttl=3600)
    return user

# Pattern 2: Write-Through
def update_user(user_id, data):
    # Update database
    db.update("users", user_id, data)

    # Update cache immediately
    cache.set(f"user:{user_id}", data)

```

Component 5: Queue (Async Layer)

Vai trò: Decouple services, enable async processing

Why queues?

- Long-running tasks (don't block user)
- Traffic spikes (buffer requests)
- Service decoupling (sender doesn't need receiver online)
- Retry logic (handle failures)

Common implementations:

- RabbitMQ (message broker)
- Kafka (event streaming)
- AWS SQS (managed queue)
- Redis (simple queue)

Characteristics:

- Asynchronous (fire and forget)
- At-least-once delivery (usually)
- Ordered or unordered
- Persistent or in-memory

Key principle: Use for work that can be delayed

Example:

```

# API: Quick response to user
@app.post("/orders")
def create_order(order_data):
    order = db.create_order(order_data)

    # Don't send email synchronously (slow!)
    # Put in queue instead
    queue.publish("order.created", {
        "order_id": order.id,
        "user_email": order_data.email
    })

    return 201, {"order_id": order.id}
    # User gets response immediately!

# Worker: Process async
@worker.task("order.created")
def on_order_created(message):
    # This runs in background
    send_confirmation_email(message.user_email)
    update_inventory(message.order_id)
    notify_warehouse(message.order_id)

```

Component 6: Storage (File Layer)

Vai trò: Store large files (images, videos, documents)

Why separate from database?

- Databases bad for large files
- Need scalability (terabytes/petabytes)
- Need CDN integration
- Cost optimization

Common implementations:

- S3 (AWS object storage)
- Google Cloud Storage
- Azure Blob Storage
- MinIO (self-hosted S3-compatible)

Characteristics:

- Cheap (compared to database)
- Scalable (unlimited storage)
- Durable (99.99999999% durability)
- Slow for small files (latency overhead)

Key principle: Store metadata in DB, files in object storage

Example:

```

# Upload flow
@app.post("/upload")
def upload_photo(file):
    # 1. Generate unique filename
    filename = f"{uuid.uuid4()}.jpg"

    # 2. Upload to S3
    s3_url = s3.upload(filename, file)

    # 3. Save metadata to database
    photo = db.create({
        "filename": filename,
        "s3_url": s3_url,
        "size": file.size,
        "uploaded_at": datetime.now()
    })

    return {"photo_id": photo.id, "url": s3_url}

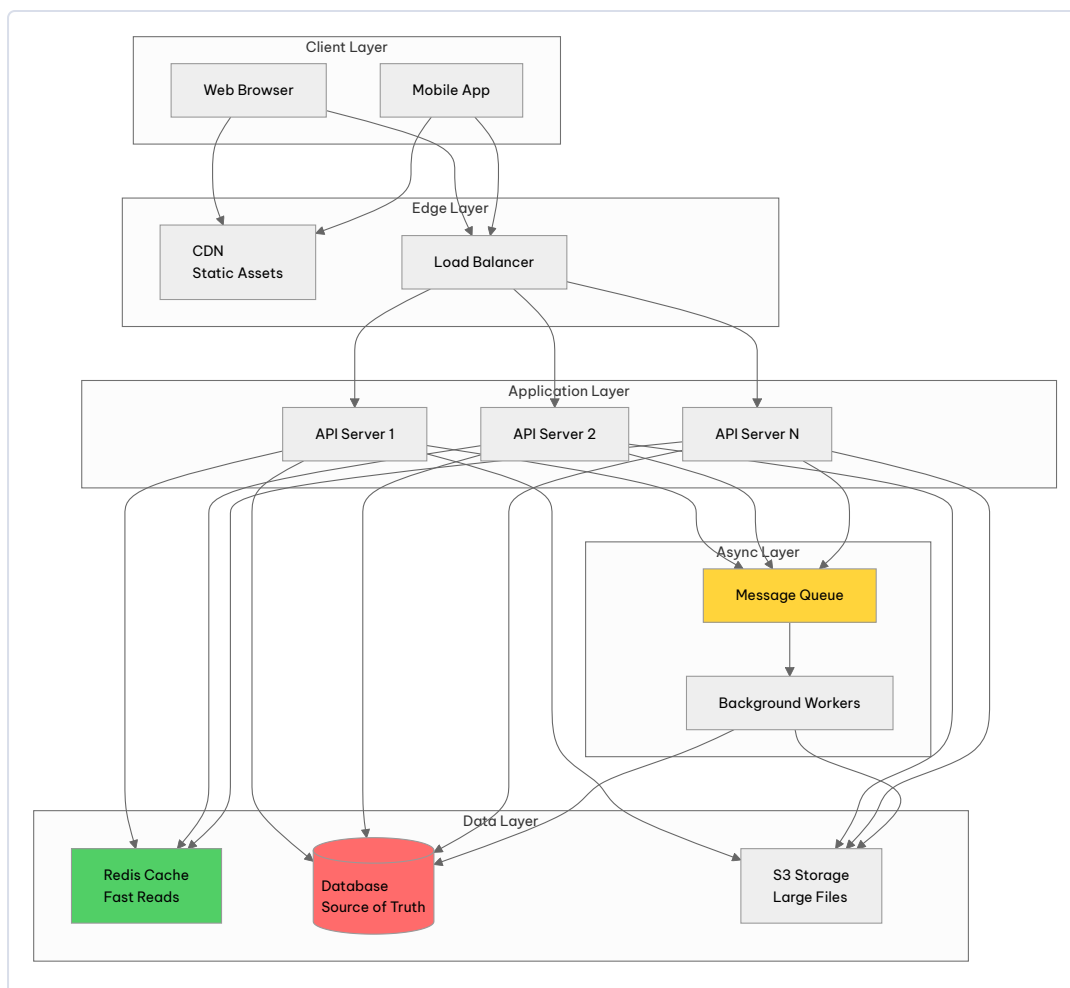
# Retrieve flow
@app.get("/photos/{id}")
def get_photo(id):
    # Get metadata from database
    photo = db.get_photo(id)

    # Client downloads from S3 directly
    return {"url": photo.s3_url}

```

Putting It All Together: System Architecture

Typical web application architecture:



Complete system: Tất cả components work together

Request Flow Example: User Updates Profile

Trace một request qua toàn bộ system:

Step 1: User clicks "Save Profile"
Component: Client (Browser)
Action: POST /users/123 with new data

Step 2: Request hits Load Balancer
Component: Load Balancer
Action: Route to API Server 2 (least connections)
Time: +10ms

Step 3: API validates and processes
Component: API Server
Actions:

- Validate input (email format, etc.)
- Check permissions (is user allowed?)
- Sanitize data

Time: +20ms

Step 4: Update database
Component: Database
Actions:

- Execute UPDATE query
- Transaction commit
- Confirm success

Time: +100ms

Step 5: Invalidate cache
Component: Cache
Actions:

- Delete cache key "user:123"
- Force fresh read next time

Time: +5ms

Step 6: Queue async tasks
Component: Queue
Actions:

- Publish "user.updated" event
- Non-blocking (returns immediately)

Time: +2ms

Step 7: Return response to user
Component: API → Client
Response: 200 OK
Total time: 137ms (good UX!)

Step 8: Background processing
Component: Worker (async)
Actions:

- Send "profile updated" email
- Update search index
- Sync to analytics

Time: Doesn't affect user experience

Key observations:

- ✓ User gets response in 137ms (fast!)
- ✓ Heavy work done async (email, indexing)
- ✓ Cache invalidated (consistency)
- ✓ Each component has clear role
- ✓ Failures can be isolated

This is system thinking in action!

The Mental Model: Input → Processing → Storage → Output

Apply to everything:

Example 1: E-commerce Checkout

Input:

- User clicks "Place Order"
- Cart items, shipping address, payment

Processing:

- Validate inventory
- Calculate total (items + tax + shipping)
- Process payment
- Generate order ID

Storage:

- Save order to database
- Store payment receipt in S3
- Cache order details

Output:

- Confirmation page to user
- Email confirmation (async via queue)
- Notification to warehouse (async)

Example 2: Social Media Post

Input:

- User writes post + uploads photo

Processing:

- Sanitize text (remove bad words)
- Resize/compress photo
- Generate post ID

Storage:

- Save post to database
- Upload photo to S3
- Cache hot posts (trending)

Output:

- Show post to user
- Fanout to followers (async via queue)
- Update feed cache (async)

Example 3: Video Upload

Input:

- User uploads video file

Processing:

- Validate file (format, size)
- Generate thumbnail
- Queue encoding job

Storage:

- Save original to S3
- Save metadata to database
- Store encoding job in queue

Output:

- Show "Processing..." to user
- Email when ready (async)
- Multiple resolutions in S3 (async)

Pattern is consistent across all systems!

Common System Patterns

Pattern 1: Read-Heavy System

Characteristics:

- 90%+ reads, < 10% writes
- Examples: News site, documentation, social media feed

Optimization:

- Aggressive caching (Redis)
- CDN for static content
- Database read replicas
- Cache-first architecture

Architecture:

Client → CDN (90% hits) → Load Balancer → API → Cache (95% hits) → Database (5% hits)

Result: 99.5% requests never hit database!

Pattern 2: Write-Heavy System

Characteristics:

- 50%+ writes
- Examples: Analytics, logging, IoT sensors

Optimization:

- Write buffering (queue)
- Batch processing
- Write-optimized database (Cassandra)
- Async everything

Architecture:

Client → API → Queue → Worker → Database (batched writes)

Result: Handle 100K writes/sec with buffering

Pattern 3: Real-Time System

Characteristics:

- Low latency critical (< 100ms)
- Examples: Chat, gaming, trading

Optimization:

- WebSocket connections
- In-memory everything
- Geographic distribution
- Minimal processing

Architecture:

Client ↔ WebSocket Server → Redis (pub/sub) → Database (eventual)

Result: Sub-50ms message delivery

Debugging with System Thinking

When production issue happens:

Before System Thinking:

```
"API is slow. Let me check the code..."
*Reads 1000 lines of code*
*No idea where problem is*
```

After System Thinking:

"API is slow. Let me trace the request:

1. Client → API: 50ms (normal)
2. API → Cache: 2ms (normal)
3. API → Database: 5000ms (PROBLEM!)
4. Database → API: 2ms
5. API → Client: 10ms

Bottleneck: Database query taking 5 seconds

Next: Check database

- Slow query log
- Missing index?
- Lock contention?

Found: Missing index on user_id

Fix: CREATE INDEX idx_user_id

Result: Query now 50ms"

→ Solved in 10 minutes!

System thinking = Structured debugging approach

Key Takeaways

System = Components + Interactions

6 core components:

1. Client (input layer)
2. API (processing layer)
3. Database (storage layer)
4. Cache (performance layer)
5. Queue (async layer)
6. Storage (file layer)

Each has specific role

Together form complete system

Universal pattern:

Input → Processing → Storage → Output

Every system follows this

Understand pattern → Understand systems

System thinking benefits:

- ✓ See the big picture
- ✓ Identify bottlenecks faster
- ✓ Design better architectures
- ✓ Debug production issues
- ✓ Communicate with team
- ✓ Make informed trade-offs

Mental shift:

From: "How does this function work?"
To: "How does request flow through system?"

From: "What does this code do?"
To: "What is the role of this component?"

From: "Fix this bug"
To: "Which component is causing issue?"

Remember:

Code is implementation detail
System is the big picture

Master code → Good developer
Master systems → Good architect

You need both, but system view comes first

System Question Framework - Checklist Chuẩn Khi Design Hệ Thống

Khi bắt đầu design một hệ thống, nhiều engineer mới thường nhảy thẳng vào giải pháp mà quên mất một bước cực kỳ quan trọng: **đặt câu hỏi đúng**.

Tôi từng mắc lỗi này. Design một hệ thống chat, tôi nghĩ ngay đến WebSocket, Redis Pub/Sub, horizontal scaling... rồi giữa chừng mới phát hiện: khách hàng chỉ cần 100 users online đồng thời. Tôi đã over-engineer một cách thảm hại.

Vấn đề không phải là thiếu kiến thức kỹ thuật. Vấn đề là tôi không hỏi đủ câu hỏi trước khi design.

Tại Sao Cần Framework Câu Hỏi?

Khi thiết kế hệ thống, có hàng trăm thứ cần cân nhắc. Không ai có thể nhớ hết. Bạn cần một **checklist** để đảm bảo không bỏ sót điểm quan trọng nào.

Think về nó như pilot checklist trước khi cất cánh. Dù đã bay 10,000 giờ, pilot vẫn phải check từng bước một. Tại sao? Vì checklist đảm bảo consistency và an toàn.

System design cũng vậy. Framework câu hỏi là safeguard của bạn.

7 Câu Hỏi Cốt Lõi

Đây là 7 câu hỏi tôi luôn hỏi khi bắt đầu design bất kỳ hệ thống nào. Order có ý nghĩa — nó đi từ context tổng thể đến chi tiết kỹ thuật.

1. Scale: Hệ Thống Sẽ Lớn Đến Đâu?

Đây là câu hỏi đầu tiên vì lý do đơn giản: **architecture cho 1,000 users hoàn toàn khác architecture cho 10 triệu users**.

Questions cụ thể:

- Bao nhiêu users hiện tại?

- Expected growth trong 1-2 năm?
- Peak concurrent users?
- Số lượng transactions/requests per day?

Tại sao quan trọng: Scale quyết định technology choices. Với 1,000 users, một monolith + MySQL là quá đủ. Với 10 triệu users, bạn cần microservices, caching layers, database sharding.

Ví dụ thực tế: Một startup với 5,000 users không cần Kubernetes và distributed cache. Nhưng nếu họ expect growth 50x trong năm tới, bạn cần design với headroom.

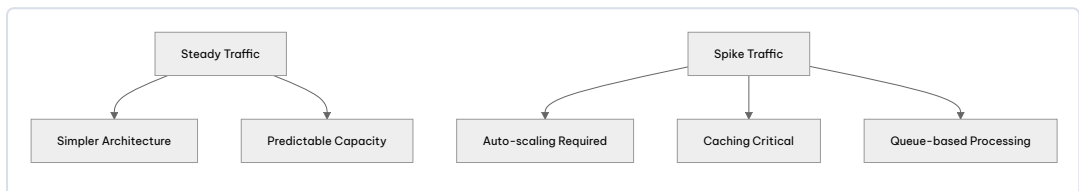
2. Traffic Pattern: Người Dùng Hoạt Động Như Thế Nào?

Scale chỉ cho bạn con số. Traffic pattern cho bạn **behavior**.

Questions cụ thể:

- Traffic đều hay có spike?
- Peak hours nào trong ngày?
- Seasonal patterns? (Black Friday, Tết)
- Read-heavy hay write-heavy?
- Ratio giữa reads và writes?

Tại sao quan trọng: Pattern ảnh hưởng đến caching strategy, auto-scaling policies, và database design.



Ví dụ: Một hệ thống e-commerce có spike traffic vào 12h trưa và 8h tối. Bạn không thể provision cho peak load 24/7 — quá tốn kém. Bạn cần auto-scaling + aggressive caching.

Ngược lại, một internal tool với steady traffic có thể dùng fixed capacity đơn giản hơn.

3. Data Size: Bao Nhiêu Data Và Tốc Độ Tăng?

Đây là câu hỏi về storage và data lifecycle.

Questions cụ thể:

- Tổng data size hiện tại?
- Data growth rate? (GB/day, TB/month)
- Average size mỗi record?
- Data retention policy? (giữ mãi hay xóa sau X tháng?)
- Hot data vs cold data ratio?

Tại sao quan trọng: Data size quyết định database choice, storage strategy, và archival policies.

Real example: Một analytics platform sinh ra 100GB logs mỗi ngày. Sau 1 năm là 36TB. Bạn không thể query directly trên 36TB. Bạn cần:

- Hot storage cho 30 ngày gần nhất (fast SSD)
- Cold storage cho historical data (S3/Glacier)
- Data aggregation và sampling

4. Availability: Downtime Có Chấp Nhận Được Không?

Availability là về **acceptable downtime**.

Questions cụ thể:

- System có thể down không?
- Acceptable downtime: phút, giờ, hay giây?
- Cần 99.9% hay 99.99% uptime?
- Impact của downtime? (mất revenue, ảnh hưởng users)
- Maintenance window có acceptable không?

Tại sao quan trọng: Availability requirements drive architecture complexity và cost.

99.9% uptime = ~8.7 hours downtime/year → single server OK
99.99% uptime = ~52 minutes downtime/year → need redundancy
99.999% uptime = ~5 minutes downtime/year → multi-region + complex failover

Ví dụ: Internal tool có thể down maintenance 2-3 giờ đêm khuya? Design đơn giản. Payment system không được down 1 giây? Cần multi-region active-active.

Trade-off: Mỗi "9" thêm vào availability tăng complexity và cost exponentially.

5. Consistency: Dữ Liệu Phải Đúng Ngay Lập Tức?

Đây là câu hỏi về **data correctness** và **synchronization**.

Questions cụ thể:

- Strong consistency cần thiết không?
- Eventual consistency có chấp nhận được?
- Stale data trong vài giây có OK?
- Conflict resolution strategy?
- Transaction guarantees cần đến đâu?

Tại sao quan trọng: Consistency requirement ảnh hưởng database choice và architecture patterns.

Examples:

- **Strong consistency cần thiết:** Banking transactions, inventory systems
- **Eventual consistency OK:** Social media feeds, analytics dashboards, view counts

Trade-off class: Bạn không thể có cả strong consistency VÀ high availability VÀ partition tolerance (CAP theorem). Phải chọn.

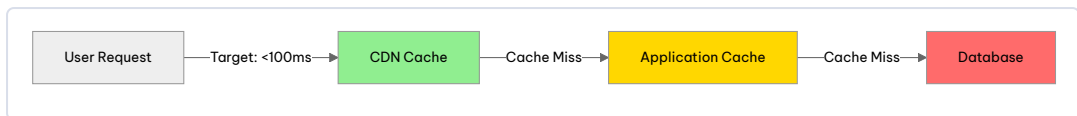
6. Latency: Nhanh Đến Mức Nào?

Latency là về **user experience** và **response time**.

Questions cụ thể:

- Acceptable response time? (ms, seconds)
- P50, P95, P99 latency requirements?
- User experience expectations?
- Real-time hay batch processing OK?
- Geographic distribution của users?

Tại sao quan trọng: Latency requirements drive caching strategy, database indexing, và CDN usage.



Ví dụ:

- Search engine: <100ms critical cho UX
- Batch report generation: vài phút OK
- Real-time chat: <500ms expected
- Analytics dashboard: 2-3 seconds acceptable

P99 latency quan trọng hơn average. Nếu 99% requests nhanh nhưng 1% slow, user experience vẫn tệ.

7. Cost: Budget Và Constraints Gì?

Cuối cùng nhưng cực kỳ quan trọng: **cost constraints**.

Questions cụ thể:

- Monthly infrastructure budget?
- Cost per user acceptable?
- Build vs buy trade-offs?
- Team size và expertise?
- Operational cost (maintenance, monitoring)?

Tại sao quan trọng: Best technical solution không có ý nghĩa nếu vượt budget hoặc team không maintain được.

Real scenarios:

- Startup với limited budget: optimize for cost, dùng managed services
- Enterprise với compliance: có thể spend more cho security và reliability
- High-growth company: optimize for speed to market, cost secondary

Cost không chỉ là infrastructure. Gồm:

- Engineering time (build và maintain)

- Third-party services
- Infrastructure (servers, databases, storage)
- Operational overhead (monitoring, on-call)

Framework Trong Thực Tế

Khi design một hệ thống, tôi follow flow này:



Quy trình cụ thể:

1. **Listen first** — nghe requirement ban đầu
2. **Ask systematically** — đi qua 7 câu hỏi theo thứ tự
3. **Clarify ambiguity** — không assume, hỏi rõ
4. **Document answers** — viết ra, không rely on memory
5. **Identify conflicts** — spot contradictions sớm
6. **Discuss trade-offs** — transparent về limitations

Common Mistakes Khi Hỏi

Qua kinh nghiệm, đây là những lỗi phổ biến:

1. Hỏi không đủ deep

❌ "Scale bao nhiêu?" — quá vague ✅ "Bao nhiêu concurrent users peak hour? Growth rate expected?"

2. Accept câu trả lời mơ hồ

❌ User: "Cần highly available" ✅ Follow-up: "Highly available nghĩa là bao nhiêu downtime acceptable? 1 hour/year hay 1 minute/year?"

3. Không challenge unrealistic requirements

Nếu user muốn: strong consistency + 99.999% availability + <10ms latency + \$100/month budget → không realistic. Bạn phải point out và discuss trade-offs.

4. Quên hỏi về constraints

Đừng chỉ hỏi về goals. Hỏi về limitations: budget, team size, compliance requirements, legacy systems.

Practical Example: Design URL Shortener

Áp dụng framework:

Scale:

- 100M URLs created/month
- 1B clicks/month (10:1 read-write ratio)

Traffic Pattern:

- Steady traffic, slight spike business hours
- Read-heavy (clicks >> creates)

Data Size:

- ~10 bytes/shortened URL
- $100\text{M} \times 10 \text{ bytes} = 1\text{GB/month}$
- Negligible growth

Availability:

- 99.9% OK (short downtime acceptable)
- Not mission-critical

Consistency:

- Eventual consistency OK for analytics
- Strong consistency for URL creation (no duplicates)

Latency:

- Redirects: <100ms critical
- Creation: <500ms OK

Cost:

- Startup budget: optimize for cost
- Managed services preferred

Decisions dựa trên answers:

- Single database OK (1GB data size nhỏ)
- Aggressive caching cho reads (10:1 ratio)
- Simple architecture (99.9% availability enough)
- Use managed database (cost-optimize, small team)

Key Takeaways

System Question Framework không phải magic formula. Nó là **thinking tool** giúp bạn structure conversation và không miss important factors.

Remember:

Always start with questions — đừng jump vào solution ngay **Ask systematically** — follow checklist để không miss gì **Clarify vague answers** — push for specific numbers **Document everything** — write down để reference sau **Identify trade-offs early** — conflicts thường emerge từ answers **Be pragmatic** — perfect solution không tồn tại, chỉ có good-enough solution given constraints

Framework này không guarantee perfect design. Nhưng nó guarantee bạn **asked the right questions** trước khi design — và đó là điểm khởi đầu quan trọng nhất.

Next time bạn design system, pull out checklist này. Đi qua từng câu hỏi một. Bạn sẽ surprised về bao nhiêu assumption bạn đã tránh được và bao nhiêu trade-off bạn identify sớm.

That's the power of good questions.

Mental Model Recap - củng cố Tư Duy System Design

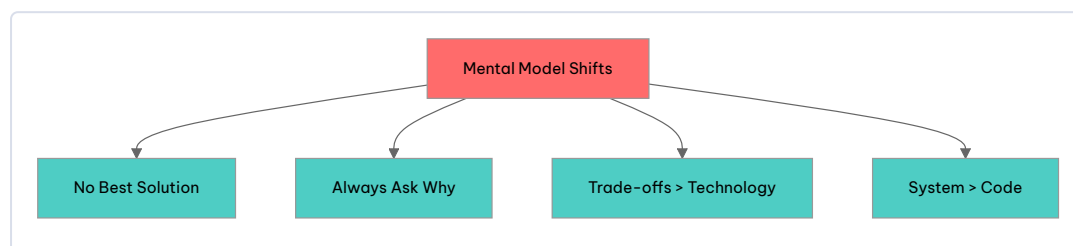
Chúng ta đã đi qua 7 lessons trong Phase 0. Đây không phải là phase dạy technology hay design patterns. Đây là phase **thay đổi cách bạn think**.

Trước khi move sang Phase 1, tôi muốn pause và recap lại những mental shifts quan trọng nhất. Những shifts này sẽ theo bạn trong suốt journey từ đây.

Think về nó như foundation của một tòa nhà. Bạn có thể học hết các công cụ, patterns, technologies — nhưng nếu foundation sai, everything else sẽ shaky.

The Four Core Mindsets

Phase 0 xoay quanh 4 mindset shifts cốt lõi. Đây không phải checklist để học thuộc. Đây là **lenses** để nhìn problems.



Hãy đi sâu vào từng mindset.

1. Không Có "Best Solution"

Đây là shift khó nhất để accept, đặc biệt khi bạn mới transition từ developer.

The old mindset: "Có một giải pháp tốt nhất. Tôi cần tìm nó."

The new mindset: "Mọi solution đều có trade-offs. Tôi cần tìm solution phù hợp nhất với context này."

Tại Sao Mindset Này Quan Trọng?

Khi bạn tin có "best solution", bạn sẽ:

- Waste time tìm kiếm perfection
- Copy solutions không phù hợp context
- Rigid khi requirements change
- Miss opportunities để optimize cho specific needs

Khi bạn accept "no best solution", bạn sẽ:

- Focus vào understanding constraints
- Design for current context
- Flexible khi adapt
- Make conscious trade-offs

Real Example

Câu hỏi: "Database nào best cho application của tôi?"

Wrong approach: "PostgreSQL là best vì X, Y, Z"

Right approach: "Depends on your needs:"

- Relational data với ACID? → PostgreSQL
- Document với flexible schema? → MongoDB
- High-throughput writes? → Cassandra
- Simple key-value? → Redis

Each có trade-offs. Best choice depends on YOUR constraints."

2. Always Ask "Why"

Mindset shift này về **questioning everything** — không accept answers at face value.

The old mindset: "Senior engineer nói dùng microservices → tôi dùng microservices"

The new mindset: "Tại sao microservices? Problem nào nó solve? Trade-offs gì? Có alternatives?"

Why "Why" Matters

Trong system design, mọi decision đều có ripple effects. Nếu bạn không hiểu **reasoning** behind decisions, bạn sẽ:

- Repeat mistakes của người khác
- Không adapt được khi context changes
- Make inconsistent decisions
- Không defend được choices của mình

The Five Whys Technique

Khi ai đó suggest một solution, drill down:

"Chúng ta nên dùng Redis."

→ Why Redis?

"Để cache data."

→ Why cache? Performance issue gì?

"API response time ~2 seconds."

→ Why slow? Database query chậm?

"Có, query scan whole table."

→ Why không add index trước?

"Ah... chưa thử index."

Thường thì sau 2-3 "why", bạn discover simpler solution.

Apply To Everything

- Why microservices instead of monolith?
- Why NoSQL instead of SQL?
- Why Kubernetes instead of simple VMs?
- Why event-driven instead of request-response?
- Why cache this data?
- Why split this service?

Không có câu hỏi "stupid". Mọi "why" đều valid.

3. Trade-offs Quan Trọng Hơn Technology

Đây là shift from **technology-first** sang **problem-first thinking**.

The old mindset: "Technology X hot và mới. Tôi muốn dùng X."

The new mindset: "Problem là gì? Trade-offs nào matter? Technology nào fit best cho trade-offs này?"

Why This Matters

Technology is just a tool. Choosing technology WITHOUT understanding trade-offs giống như:

- Mua Ferrari để đi chợ hàng ngày
- Dùng sledgehammer để đóng đinh
- Over-engineer cho problems đơn giản

Good architects think về trade-offs FIRST, technology SECOND.

Common Trade-offs

Mọi design decision involve trade-offs:

Consistency vs Availability:

- Strong consistency → có thể sacrifice availability
- High availability → có thể accept eventual consistency

Performance vs Cost:

- Better performance → more expensive infrastructure
- Lower cost → accept slower response times

Simplicity vs Flexibility:

- Simple architecture → harder to extend
- Flexible architecture → more complexity upfront

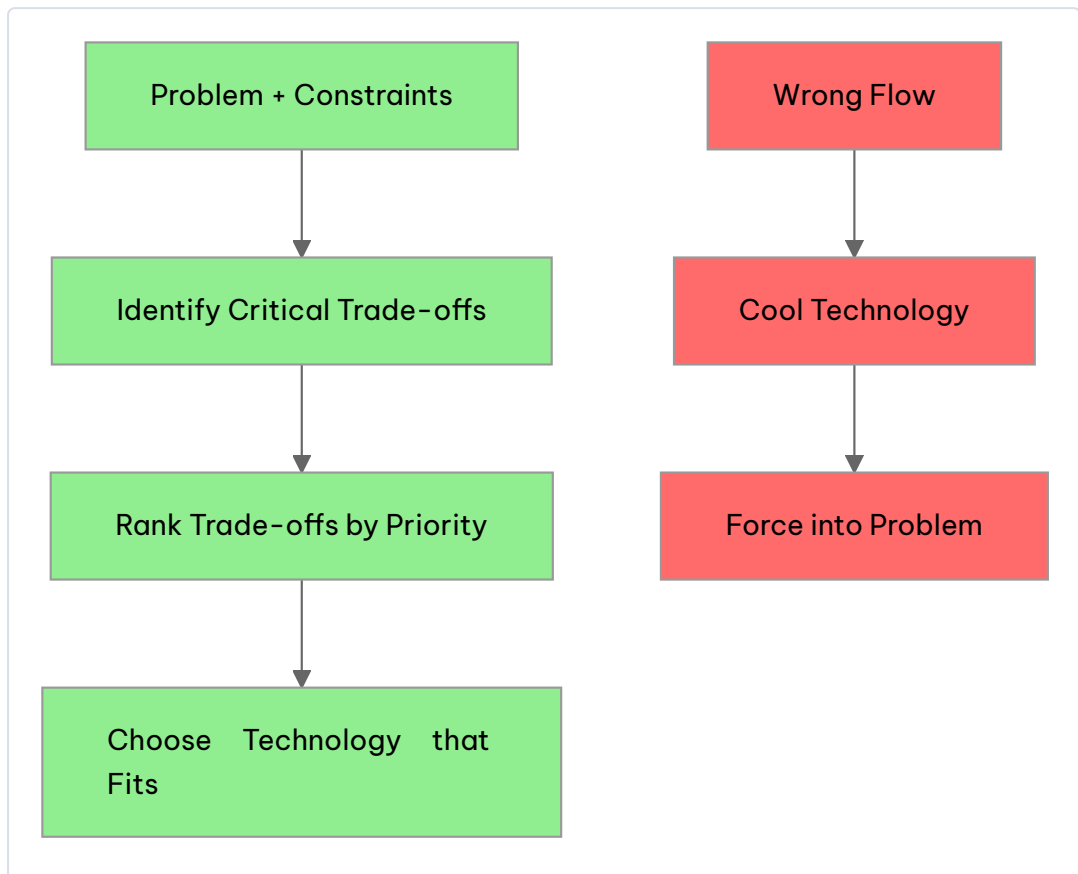
Build vs Buy:

- Build custom → full control, higher cost
- Use managed services → less control, faster delivery

Framework Thinking

Thay vì think "technology nào?", think:

1. Trade-off nào critical cho problem này?
2. Trade-off nào acceptable?
3. Technology nào support trade-offs đó?



4. System Thinking Trên Code Thinking

Đây là biggest shift — from **micro level** (code) to **macro level** (system).

The old mindset: "Tôi viết clean code, optimize algorithms, follow best practices. That's enough."

The new mindset: "Code chỉ là một phần. Tôi cần think về: components interaction, data flow, failure modes, scaling, monitoring, operations."

Why This Shift Is Hard

Developers spend years optimizing code:

- Clean architecture
- Design patterns
- Algorithm optimization
- Code quality

Suddenly, system design nói: "Code quality ít quan trọng hơn system quality."

Điều này không có nghĩa code không quan trọng. Nhưng **good code trong bad architecture vẫn fail**.

What Is System Thinking?

System thinking means nhìn **whole picture**, không chỉ individual parts:

Code thinking:

- Function này $O(n^2)$ → optimize to $O(n \log n)$
- Class này violate SOLID → refactor

System thinking:

- Service này become bottleneck at scale → cần split hay scale?
- Database này single point of failure → cần replication?
- Network latency between services → có nên merge services?
- Failure ở component A affect component B như thế nào?

Example: Payment Processing

Code-level thinking:

```
PaymentService.process(payment):  
  - Validate payment data  
  - Call payment gateway  
  - Update database  
  - Return response
```

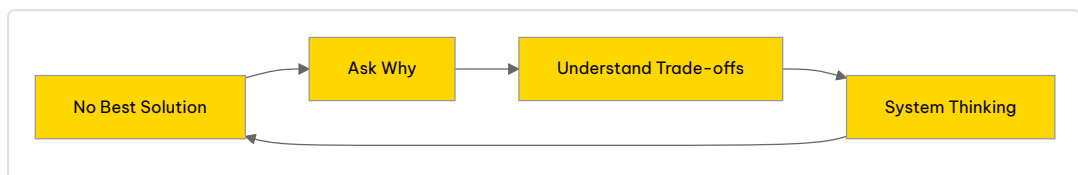
System-level thinking:

1. What if payment gateway timeout?
→ Retry? Queue? Idempotency?
2. What if database down khi update?
→ Đã charge customer nhưng chưa save record?
3. How to handle high load?
→ Queue-based processing? Rate limiting?
4. How to monitor failures?
→ Metrics? Alerts? Dashboards?
5. How to debug issues in production?
→ Logging? Tracing? Correlation IDs?

Code thinking focus vào logic. System thinking focus vào **reliability, scalability, observability**.

Kết Nối 4 Mindsets

4 mindsets này không isolated. Chúng interconnected:



Flow tự nhiên:

1. Accept **không có best solution** → bạn open-minded
2. **Ask why** → bạn understand reasoning
3. Focus vào **trade-offs** → bạn make informed decisions
4. Apply **system thinking** → bạn see big picture

Mỗi mindset reinforce mindsets khác.

From Mindset To Practice

Okay, bạn understand 4 mindsets. Bây giờ làm sao apply?

Daily Practices

1. Review Past Decisions

Pick một architectural decision trong project của bạn. Ask:

- Tại sao decision đó?
- Trade-offs gì?
- Với hindsight, còn đúng không?
- Học được gì?

2. Challenge Yourself

Mỗi khi bạn muốn dùng một technology:

- Write down 3 reasons why
- Write down 2 alternatives
- Compare trade-offs
- Justify choice

3. Think In Systems

Khi debug issue trong production:

- Đừng chỉ fix bug
- Ask: tại sao bug này xảy ra?
- System design nào prevent nó?
- Có issues tương tự khác không?

4. Learn From Others

Đọc architecture blogs, case studies:

- Không copy solutions
- Focus vào reasoning
- Understand their constraints
- Note trade-offs họ chọn

Red Flags To Watch

Dấu hiệu bạn chưa internalize mindsets:

"Best practice là X, nên tôi dùng X" → Missing: context và trade-offs

"Senior engineer nói như vậy" → Missing: asking why

"Technology Y hot, tôi muốn learn" → Missing: problem-first thinking

"Code này clean và fast" → Missing: system-level concerns

Khi bạn catch yourself thinking như vậy — pause và apply framework.

Key Takeaways

Nếu chỉ nhớ 4 điều từ Phase 0:

- 1. No Best Solution** → Context determines "best". Focus on fit, not perfection.
- 2. Always Ask Why** → Understanding reasoning > memorizing solutions.
- 3. Trade-offs > Technology** → Choose based on constraints, not trends.
- 4. System > Code** → Good code in bad architecture still fails.

Final Thought

Mindset shift không xảy ra overnight. Tôi mất 2 năm để truly internalize những principles này.

Bạn sẽ catch yourself thinking theo old ways. That's normal. Mỗi lần đó, consciously apply new mindset. Overtime, nó trở thành automatic.

Remember: **System design is not about knowing all technologies. It's about thinking clearly about trade-offs trong complex contexts.**

System Design Interview Preparation

Tài liệu này dành cho developer đang muốn chuẩn bị nghiêm túc cho **system design interview** ở level mid đến senior.

Mục tiêu không phải là học thuộc "template trả lời", mà là xây dựng tư duy kiến trúc có cấu trúc, giải thích được trade-off, và giao tiếp rõ ràng dưới áp lực phỏng vấn.

Ngữ cảnh lộ trình: nội dung này phù hợp nhất khi bạn đã đi qua nền tảng từ Phase 0 đến Phase 5 và đang tiến vào mindset của **Phase 6 — System Design Mastery**.

1) System Design Interview là gì?

System Design Interview kiểm tra cách bạn tư duy khi đứng trước một bài toán hệ thống mở, thiếu dữ kiện, có nhiều ràng buộc thực tế.

Mục đích chính

- Đánh giá khả năng **thiết kế hệ thống** chứ không chỉ viết code.
- Đánh giá cách bạn ra quyết định trong điều kiện không hoàn hảo.
- Đánh giá khả năng cân bằng giữa product, kỹ thuật, và vận hành.

Khác gì với coding interview?

- Coding interview: tập trung thuật toán, correctness, implementation chi tiết.
- System design interview: tập trung kiến trúc tổng thể, scale, reliability, trade-off, và communication.

Trọng tâm

Bạn được kỳ vọng thể hiện tư duy của một kỹ sư có thể thiết kế hệ thống production, không chỉ là người hoàn thành một function.

2) Interviewer thực sự đánh giá điều gì?

1. Requirement clarification

- Bạn có biết hỏi đúng câu hỏi để làm rõ scope không?
- Bạn có phân biệt functional và non-functional requirement không?

2. Trade-off reasoning

- Bạn có giải thích được vì sao chọn phương án A thay vì B?
- Bạn có nhìn thấy chi phí vận hành và độ phức tạp đi kèm không?

3. Scalability thinking

- Bạn có biết đặt giả định về QPS, data size, growth không?
- Bạn có biết hệ thống sẽ vỡ ở đâu khi scale tăng?

4. Structured communication

- Câu trả lời có mạch lạc, theo thứ tự, dễ follow không?
- Bạn có chủ động dẫn dắt buổi trao đổi không?

5. Production awareness

- Bạn có nghĩ tới reliability, observability, failure handling, rollback?
 - Bạn có biết khi nào nên giữ đơn giản và khi nào phải nâng cấp kiến trúc?
-

3) Quy trình interview chuẩn (Step-by-step)

Bước 1 – Clarify requirements

- Xác định rõ bài toán cốt lõi.
- Chốt use case ưu tiên.
- Làm rõ giới hạn: latency, consistency, availability, cost.

Bước 2 – Define scale & constraints

- Ước lượng users, QPS, storage, bandwidth.
- Xác định "điểm nóng" cần ưu tiên xử lý trước.

Bước 3 – High-level architecture

- Vẽ kiến trúc mức khối lớn.
- Xác định service chính, data flow, ranh giới trách nhiệm.

Bước 4 – Data model & API

- Thiết kế entity chính, key access pattern.
- Đề xuất API core để validate tính khả thi.

Bước 5 – Deep dive vào thành phần quan trọng

- Chọn 1-2 điểm quan trọng để đào sâu: cache, queue, DB partition, consistency model...

Bước 6 – Bottleneck & failure analysis

- Chỉ ra SPOF, hotspot, failure mode.
- Nêu chiến lược giảm rủi ro và phương án dự phòng.

Bước 7 – Trade-off & evolution

- Tóm tắt điểm mạnh/yếu của phương án.
- Nêu roadmap nâng cấp hệ thống theo giai đoạn tăng trưởng.

4) Framework gợi ý: SNAKE

Một framework ngắn gọn, dễ nhớ để bạn không bị "trôi" trong interview.

S – Scope

- Bài toán là gì?
- Use case nào ưu tiên?

- Điều gì không nằm trong scope hôm nay?

N – Numbers

- Ước lượng QPS, payload, lưu trữ, peak traffic.
- Đặt số gần đúng để định hướng kiến trúc.

A – API

- Đề xuất API cốt lõi để chốt contract giữa client/server.

K – Key Components

- Dựng kiến trúc chính: LB, app, cache, DB, queue, worker, CDN...

E – Explore Deep Dive

- Đào sâu 1-2 vấn đề có rủi ro cao nhất.
- Nói rõ trade-off và kế hoạch scale tiếp theo.

Ví dụ nhanh (Design URL Shortener)

- **Scope:** tạo short URL, redirect nhanh, thống kê click cơ bản.
 - **Numbers:** 50K req/s read, 5K req/s write, 5 năm dữ liệu.
 - **API:** `POST /shorten`, `GET /{code}`.
 - **Key components:** API service + KV store + cache + analytics pipeline.
 - **Explore:** collision strategy, hot key, eventual consistency cho analytics.
-

5) Các câu hỏi interview phổ biến và thứ được kiểm tra

Design Twitter / X

Kiểm tra: fan-out strategy, feed generation, read/write trade-off, timeline consistency.

Design YouTube

Kiểm tra: upload pipeline, transcoding, CDN strategy, metadata store, recommendation boundaries.

Design Chat System

Kiểm tra: real-time delivery, connection management, ordering, offline sync, presence.

Design URL Shortener

Kiểm tra: ID generation, redirect latency, hot key handling, read-heavy optimization.

Design Notification System

Kiểm tra: multi-channel delivery, retry policy, deduplication, rate limiting, observability.

6) Cách chuẩn bị hiệu quả

Theo lộ trình học

- Củng cố foundation (Phase 0-2).
- Nâng độ sâu distributed + scale (Phase 3-4).
- Luyện pattern thực tế và decision narrative (Phase 5-6).

Chiến lược luyện tập

- 3-4 đề/tuần, mỗi đề 45-60 phút.
- 1 buổi/tuần chỉ để review và sửa cách trình bày.
- Luyện cả breadth (nhiều loại đề) và depth (đào sâu 1 đề).

Whiteboard / diagram practice

- Vẽ nhanh kiến trúc ở mức component trước.
- Giảm chi tiết thừa, tăng tính rõ ràng data flow.

Communication practice

- Nói thành tiếng khi giải đề.
- Ghi âm 1-2 lần/tuần để tự phát hiện điểm rồi.

Mock interviews

- Tập với bạn học hoặc mentor.
- Sau mỗi buổi, chốt 3 điểm mạnh và 3 điểm cần sửa.

Học từ hệ thống thật

- Đọc engineering blog (Uber, Netflix, Cloudflare, Meta...).
- Tóm tắt lại theo khung: bài toán -> giải pháp -> trade-off -> bài học.

7) Các lỗi rất thường gặp

- Nhảy vào giải pháp quá sớm khi chưa chốt requirement.
- Không hỏi số liệu scale nên kiến trúc thiếu cơ sở.
- Nói quá nhiều thành phần nhưng không giải thích trade-off.
- Over-engineering ngay từ đầu.
- Không phân biệt "must-have bây giờ" và "evolution sau".
- Bỏ qua reliability, failure handling, observability.

Checklist chống lỗi: trước khi kết thúc, luôn tự hỏi "Nếu traffic tăng 10x, hệ thống sẽ vỡ ở đâu trước?"

8) Lộ trình chuẩn bị đề xuất (8 tuần)

Tuần 1-2

- Ôn framework trả lời (SNAKE).
- Luyện 4 đề cơ bản: URL shortener, rate limiter, notification, chat basic.

Tuần 3-4

- Tăng độ sâu distributed systems: consistency, partition, queue, cache invalidation.
- Luyện 4 đề trung bình: feed, file storage, search basic, analytics pipeline.

Tuần 5-6

- Tập medium-hard: YouTube, ride-hailing dispatch overview, ad serving overview.
- Tập bottleneck/failure analysis kỹ hơn.

Tuần 7

- Mock interview cường độ cao: 3-4 buổi/tuần.
- Chấm theo rubric: clarity, structure, trade-off, depth.

Tuần 8

- Final review: top lỗi cá nhân, top pattern cần nhớ, top câu hỏi clarifying.
- Giảm tải trước interview, tập trung vào sự rõ ràng và bình tĩnh.

9) Communication tips trong interview

Think aloud có cấu trúc

Nói rõ bạn đang làm bước nào: requirement -> numbers -> architecture -> deep dive -> trade-off.

Dẫn dắt cuộc trao đổi

- Chủ động đề xuất giả định khi thiếu dữ liệu.
- Thường xuyên checkpoint: "Anh/chị muốn em đào sâu phần nào tiếp theo?"

Hỏi câu hỏi tốt

- Peak traffic hay average?
- Ưu tiên latency hay consistency?

- MVP trước hay long-term scale trước?

Khi gặp bài toán chưa từng làm

- Bình tĩnh quay về nguyên lý gốc.
 - Chia nhỏ bài toán.
 - Chọn baseline đơn giản trước rồi mở rộng.
-

10) Bắt đầu ngay

Nếu bạn muốn chuẩn bị system design interview một cách bài bản:

- Hoàn thành các phase nền tảng nếu còn thiếu.
- Vào **Phase 6 — System Design Mastery** để luyện khung tư duy interview.
- Lên lịch mock interview định kỳ từ tuần này.

Bảng thuật ngữ Anh – Việt

Các thuật ngữ chuyên ngành đã được Việt hoá trong sách. Lần đầu xuất hiện trong mỗi bài, thuật ngữ gốc được ghi kèm trong ngoặc.

Tiếng Anh	Tiếng Việt
alternatives	các phương án thay thế, phương án thay thế
api calls	lời gọi api
approach	cách tiếp cận
approaches	cách tiếp cận
architect	kiến trúc sư
async	bất đồng bộ
asynchronous	bất đồng bộ
authentication	xác thực
auto-scaling	tự động mở rộng
availability	tính sẵn sàng
background workers	tiền trình xử lý nền
bandwidth	băng thông
best practices	thực hành tốt nhất
bottlenecks	điểm nghẽn
building blocks	khối xây dựng
business logic layer	tầng nghiệp vụ
cache	bộ nhớ đệm
cache invalidation	vô hiệu hoá bộ nhớ đệm
cache layer	lớp bộ nhớ đệm
cache-aside	cache-aside
caching strategy	chiến lược lưu đệm

Tiếng Anh	Tiếng Việt
client	máy khách
code-first	lấy mã nguồn làm trung tâm
code-first thinking	tư duy ưu tiên viết mã
communication	giao tiếp
communication patterns	mẫu giao tiếp
complexity	độ phức tạp
components	thành phần
consistency	tính nhất quán
constraints	ràng buộc
context	bối cảnh
cost	chi phí
data flow	luồng dữ liệu
database	cơ sở dữ liệu
database queries	truy vấn cơ sở dữ liệu
debug	gỡ lỗi
denormalization	phi chuẩn hoá
deploy	triển khai
developer	lập trình viên
developers	lập trình viên
distributed cache	bộ nhớ đệm phân tán
distributed system	hệ thống phân tán
downtime	thời gian ngừng hoạt động
eventual consistency	nhất quán cuối cùng
fanout	phân phát
feed	bảng tin
framework	khung tư duy, khung phát triển

Tiếng Anh	Tiếng Việt
high availability	tính sẵn sàng cao
historical data	dữ liệu lịch sử
horizontal scaling	mở rộng theo chiều ngang
indexes	chỉ mục
infrastructure	hạ tầng
junior engineer	kỹ sư mới vào nghề
latency	độ trễ
load	tải
load balancer	bộ cân bằng tải
loose coupling	liên kết lỏng
maintain	bảo trì
memory	bộ nhớ
mental model	mô hình tư duy
mental models	mô hình tư duy
message queue	hàng đợi thông điệp
metrics	số liệu đo
microservices	vi dịch vụ
migrate data	di chuyển dữ liệu
mindset	tư duy
monitor	giám sát
monolith	hệ thống nguyên khối
network	mạng
network partition	phân vùng mạng
normalization	chuẩn hoá
optimize	tối ưu
over-engineering	thiết kế thừa

Tiếng Anh	Tiếng Việt
pattern	mẫu thiết kế
peak hours	giờ cao điểm
performance	hiệu năng
presentation layer	tầng hiển thị
problem-first thinking	tư duy ưu tiên vấn đề
production	môi trường thực tế
queue	hàng đợi
rate limit	giới hạn tần suất
real-time	thời gian thực
refactor	tái cấu trúc
reliability	độ tin cậy
request	yêu cầu
resharding	phân mảnh lại
responses	phản hồi
retry	thử lại
reverse engineering	phân tích ngược
scalability	khả năng mở rộng
scale	quy mô, mở rộng
senior architect	kiến trúc sư lâu năm
server	máy chủ
services	dịch vụ
session	phiên
sharding	phân mảnh dữ liệu
solution-first thinking	tư duy ưu tiên giải pháp
source of truth	nguồn dữ liệu gốc
spikes	đột biến tải

Tiếng Anh	Tiếng Việt
stale data	dữ liệu cũ
startup	công ty khởi nghiệp
stateful	có trạng thái
stateless	phi trạng thái
static assets	tài nguyên tĩnh
strong consistency	nhất quán mạnh
sync	đồng bộ
synchronous	đồng bộ
system design	thiết kế hệ thống
system thinking	tư duy hệ thống
target	mục tiêu
tech stack	bộ công nghệ
thinking in constraints	tư duy theo ràng buộc
throughput	thông lượng
tight coupling	liên kết chặt
time to live	thời gian sống
timeout	hết thời gian chờ
trade-off	sự đánh đổi
trade-off thinking	tư duy đánh đổi
trade-offs	đánh đổi
traffic	lưu lượng
traffic spikes	tăng đột biến lưu lượng
transactions	giao dịch
trending	xu hướng
under-engineering	thiết kế thiếu
vertical scaling	mở rộng theo chiều dọc